ARTICLE | Article in Press

Concurrency aware artificial intelligence for real-time intrusion detection in software-defined networks

R. Kanimozhi · V. Padmavathi · M. Latha

Received: 27 December 2025 / Accepted: 6 October 2026
© The Author(s) 2026

Abstract

Contemporary software systems are now being used in environments that are highly concurrent, data-intensive, and latency-sensitive. Software-Defined Networks (SDNs) applications like real-time intrusion detection need fast pattern recognition in large amounts of data streams through time. Although deep learning models have impressive detection capabilities, failure to perform well in real-world deployments is usually limited by poor concurrency management, inflexible scheduling, centralized learning pipelines, and fault tolerance. In this study, the authors suggest a concurrency-aware artificial intelligence (CA-AI) framework that directly incorporates actor-based concurrency control, workflow-based load-aware scheduling, deep long short-term memory (LSTM)-based temporal learning, agent-based parallel feature extraction, and feedback loops based on monitoring. The framework is evaluated for SDN-oriented intrusion detection workloads, where concurrency management is implemented at the system architecture level through actor-based execution, workflow scheduling, and asynchronous inference and tested on real network traffic data samples (NSL-KDD, CIC-IDS2017, and UNSW-NB15). Experimental results demonstrate that concurrency-aware system design leads to a significant improvement in the detection accuracy achieving up to 98.9% detection accuracy across multiple benchmark datasets, including NSL-KDD, CIC-IDS2017, and UNSW-NB15, throughput improved by 2.1 × compared with the centralized IDS baseline, while throughput increased by approximately 1.19 × when concurrency increased from 10,000 to 100,000 flows within the proposed CA-AI framework, average latency was reduced by approximately 57.5% compared with the centralized IDS baseline (146 ms to 62 ms), while maintaining high reliability under concurrent workloads. The proposed framework establishes concurrency as a fundamental architectural principle for scalable AI-driven pattern recognition systems.

Keywords Concurrency-aware AI · Actor model · Workflow scheduling · LSTM · Pattern recognition · Intrusion detection · Software-defined networks

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.



https://doi.org/10.1038/s41598-026-75362-x

Published online: 21 October 2026

scientific reports

Introduction

The current software systems have been developing into cloud-native, edge-enabled, and micro service-enabled architecture, which has led to unprecedented amounts of concurrency. Applications that are security critical, such as intrusion detection systems (IDS), are expected to constantly examine the large-scale network traffic and act in response to intrusions in real time. The initial methods of securing the data that was to be saved on the clouds focused on encryption before transfer which formed the foundation of subsequent intelligent and flexible security frameworks[1]. The presence of high data velocity, temporal dependencies, and workload variability creates serious issues for the traditional AI-based solutions. Deep learning-based models, including LSTM networks, have been shown to be very effective detectors of sophisticated and elusive attacks. Nevertheless, the majority of the existing IDS implementation is concurrency-blind and depends on shared memory, thread-based parallelism, and a synchronous model of execution. These solutions tend to be contended, are not scalable, and exhibit cascading failures due to realistic workloads. The present study demonstrates that real-time systems cannot utilize only pattern recognition accuracy. The management of concurrency, scheduling, reliability, and monitoring are to be co-designed together with learning models instead.

In spite of their popularity, the classical IDS architecture suffers devastating performance even in high-concurrent scenarios. Empirical studies have reported significant throughput degradation and increased latency in centralized and thread-based IDS implementations due to lock contention and synchronization overheads[2]. Prior work in real-time and distributed systems has shown that bursty traffic conditions can lead to increased deadline misses and unstable latency behaviour in thread-based architectures[3]; this results in delayed or missed intrusion responses. These constraints lead to the fact that accuracy of classification is not a sufficient parameter SDN-oriented intrusion detection environment, where the concurrency, latency, and reliability constraints are as crucial. Stream-processing frameworks such as Apache Spark Streaming and Apache Flink introduce additional latency due to micro-batching, checkpoint synchronization, and centralized coordination mechanisms[4]. Concurrency bottlenecks arising from shared-memory synchronization and blocking operations have been widely reported to limit scalability in high-throughput distributed systems[5].

Recent stream processors like Apache Spark streaming and Apache Flink are scalable data-parallel execution engines that are commonly deployed to analyse network traffic[4]. These systems are, however, more oriented towards batch-based or micro-batch processing and are based on shared state abstractions and centralized job schedulers[6]. Because of this fact, they tend to add extra latency with windowing delays, synchronization of the checkpoints, and global coordination, which restricts the usability in per-flow, low-latency intrusion detection. The suggested CA-AI framework, in contrast, is an actor-based, per-flow concurrency paradigm with asynchronous communication in all places with fine-grained isolation, lock-free execution, and fast fault recovery. This architectural difference enables CA-AI to maintain stable throughput growth under increasing concurrency and predictable and bounded latency within the evaluated experimental environment under severe concurrency, which is only hard to implement with general-purpose stream-processing systems.

Research gaps and scientific challenges in concurrency-aware AI

Although deep learning-based intrusion detection has made a considerable advancement, a number of fundamental challenges have not been fully covered in the current research. The majority of studies mainly maximize classification accuracy and implicitly, they assume that computational environments do not change. But, Software-Defined Network (SDN) environments operating in real time are dynamic systems, high-concurrency systems, and systems sensitive to latency[7]. In this case, AI systems should meet at the same time the requirements associated with scalability, fair scheduling, predictable latency behavior within the evaluated environment, and fault isolation.

The current studies of AI-based IDS have a critical failure in the architecture decoupling between learning models and concurrency management[8]. Deep networks like LSTM networks are commonly run on thread-based infrastructures or shared-memory infrastructures that experience lock contention, blocking synchronization and priority inversion at workload intensity. These bottlenecks cause degradation in throughput and deadline breakdown, as well as cascading performance breaks, as the level of concurrency grows. Therefore, high levels of classification accuracy do not always result in a high degree of real-time detection. Moreover, there is an added constraint of execution in LSTM-based systems. Since LSTM architectures are based on the idea of sequential temporal dependencies, the inference latency is susceptible to batching schemes, scheduling delays, and access patterns. This occurs because at centralized deployments, operations related to updating the model often because downtime or synchronization delays, which are not allowed in mission-critical settings. These drawbacks underscore the necessity of asynchronous inference pipelines and isolation-aware scheduling and models of concurrency coordinated execution.

The other open issue is the lack of formalization between risk-sensitive scheduling and AI inference[9]. The current workflow based schedulers do not rely on model generated risk scores which results in a poor focus on high-risk flows. Likewise, runtime AI feedback is not often used by concurrency-control strategies when dynamically scaling scheduling policies[10]. Thus, the main research gap does not consist in the fact that the correct models of learning are not available, but in the fact that a single framework of architecture that simultaneously co-designs concurrency management, scheduling, learning implementation, feature extraction and monitoring is unavailable. This paper fills this gap by suggesting a Concurrency-Aware Artificial Intelligence (CA-AI) model whereby concurrency is considered a fundamental architectural principle as opposed to an implementation consideration.

Contributions of this work

The Concurrency-Aware Artificial Intelligence in this work does not refer to any change of the internal learning process of the LSTM model. Rather, concurrency awareness is implemented in the system architecture by using actor-based execution, workflow-oriented scheduling, asynchronous inference management, parallel feature extraction, and monitoring-driven adaptation. Thus, the proposed framework doesn't change the mathematical formulation of the AI model, but it does focus on concurrency management in the intrusion detection architecture that runs in SDN environments.This paper is not a proposal of a new learning algorithm or concurrency primitive as such. Rather, its main input is the formal combination and adjustment of the existing methods to a coherent, concurrency-aware AI framework of real-time pattern recognition. Particularly, these contributions are.

1. The joint design of actor-based concurrency and deep temporal learning, in which per-flow actors are used to implement inference and state so that concurrency does not require sharing a memory and thus does not involve contention on shared memory.
2. A load-sensitive, risk-aware, workflow-based scheduling scheme that dynamically assigns higher priority to AI inference, depending on risk, deadlines, and runtime contention;
3. An agent-based parallel feature extraction pipeline that is compatible with actor isolation to minimize end-to-end latency;
4. A feedback loop that is driven by monitoring that links system-level measurements with scheduling and learning choices.

In contrast to traditional integration-based models where learning, time scheduling, and concurrency are viewed as decoupled layers, the CA-AI framework recognizes concurrency as a core architectural principle, which allows stable throughput growth under increasing concurrency, predictable latency behavior under the evaluated workloads, and high fault isolation with workloads of high concurrency.

Scientific novelty of CA-AI

The existing actor-based systems are mainly concerned with concurrency management and fault isolation, while the LSTM-based intrusion detection systems are concerned with temporal pattern learning. Generally, workflow scheduling frameworks focus on optimizing the allocation of resources, which is independent of the AI inference behavior. The proposed CA-AI framework is different from the above approaches, and is designed to explicitly integrate actor-based concurrency management, risk-aware workflow scheduling, asynchronous temporal inference, agent-based feature extraction, and monitoring-based adaptation into a unified framework. The scientific contribution is the co-design of the runtime concurrency management with AI inference execution to enable scalable and low-latency pattern recognition for highly concurrent workloads.

Literature review and motivation

The deep learning paradigm in network security

Deep learning models, especially Long Short-Term Memory (LSTM) networks[11,12], have been highly praised for being effective at extracting complex temporal relations, which sophisticated Intrusion Detection Systems (IDSs) require in dynamic networks such as Software-Defined Networks (SDNs)[13–15]. These techniques are still being perfected in recent literature, and the necessity to provide high-accuracy cyber threat adaptation[16]. Nevertheless, the present paper suggests that such emphasis on classification performance is not adequate for mission-critical deployments. Nevertheless, the essence of the problem lies in the fact that the traditional, rule-based concurrency control is basically flawed in dealing with dynamic and real-time resource conflict[17,18]. This has given rise to Deep Reinforcement Learning-Based Concurrency Control (DRLCC) frameworks, which utilize AI to optimize resource allocation dynamically and reduce the problem of priority inversion, resulting in considerable improvements in the time to complete the tasks in a highly asynchronous system and, in the case of a highly asynchronous system, urgent-job delay[17]. This affirms the argument of the CA-AI that even concurrency management should be intelligent.

The inherent flaw of the conventional rule-based concurrency control is that they rely on fixed coordination policies and on-premises centralized synchronization. These mechanisms are based on predetermined rules (e.g., fixed priorities, locks, or access ordering), which presuppose comparatively stable contention patterns. These fixed rules do not update dynamically in real-time in dynamic SDN environments where the flow arrival rates and resource demands change faster. This causes amplification of contention at locks, block delays, and priority inversion when used at high concurrency, which futilely leads to throughput degradation and latency constraint violation. The suggested CA-AI framework removes this weakness by substituting the fixed rule-based coordination with decentralization of the actors at the actor level and feedback scheduling, which allows real-time adaptation to resource contention without global locks or blocking synchronization.

The architectural gap: concurrency and scalability

Current IDS systems are mostly based on old and concurrency-incapable architectures, which entailed shared memory and thread-based parallelism[19]. These models present serious performance bottlenecks; they are contending able, lack scalability, and cause cascading failures when the flow volume among them is large[20]. Earlier architectural designs of adaptive data security in the cloud setting have shown the need to have smart, policy-driven security services in the context of dynamic workloads[21]. The core issues of the work are in direct relation to the issues set out in the current systems research: Actor-Based Systems: The Actor Model[19,20] has been created as a direct reaction to the necessity of simplified concurrency and fault isolation[22]. By supplying an encapsulation of state and only asynchronous communication, actors offer a well-tested mathematical model of creating

scalable and resilient distributed systems[19,23]. This is a zero-knowledge proofs-based, distributed-consensus-based paradigm that solves the problem of scalability and privacy as well as allowing autonomous agents to execute local AI inference workloads[24]. The decentralized compute marketplaces are in line with the distributed and workflow-oriented nature of the CA-AI[24]. Workflow Scheduling and Real-Time Suggests: In data-intensive applications, with the fixed schedule, deadlines would be missed and the congestion would be experienced. The requirement of load-adaptive, dynamically scheduled scheduling, taking into account task priority (high-risk traffic) and meeting deadlines, is a long-standing thread through real-time systems literature[23,24]. Recent studies in cloud and compute-continuum systems are growing up on the optimization of scheduling of not only performance but also reliability, energy consumption and security limits. The mode of learning and feature extraction proposed by the CA-AI framework is opposed to the traditional deep learning system that can be centralized as follows:

Next-Generation Temporal Models (Transformers): Although the LSTMs are successful, the trend of long, sequential data streams (such as network traffic) processing is shifting to Transformer Neural Networks (TNNs). TNNs eliminate sequential computation through the multi-head self-attention mechanism and allow parallel computation, which is much more effective for capturing long-range dependencies, resulting in faster and more efficient training, and spatiotemporal correlations are better characterized. Fault-Tolerant Micro services: On top of actor isolation, with the transition to cloud-native, containerized micro services, the research into training-free anomaly detection and predictive failure models[25] has been stimulated. These techniques exploit the fact that micro services have intrinsic modularity (such as replication and limited scope of functionality) to identify anomalies without using any static behavioural baseline, which decays rapidly. This adds fault isolation of the CA-AI framework, which concentrates on consistency of replicated behaviour to a heavier adaptive system and a more resilient system[25].

Finally, the CA-AI system brings together the proven concepts of deep temporal learning with the current super-modern architectural solutions of concurrent, distributed, and real-time systems study[17]. The CA-AI platform makes concurrency a key design feature to support production-grade performance in AI's demanding applications by specifically incorporating dynamic concurrency control[17], robust fault tolerance[25], and next-generation temporal models[23].

The progress in distributed learning and efficiency

The method of learning and feature extraction of The CA-AI framework is aligned with the traditional centralized deep learning. Pipeline: Asynchronous Parallelism: The framework can support throughput and convergence using asynchronous gradient aggregation and non-blocking inference[26,27]. This is reminiscent of more sophisticated methods in distributed reinforcement learning and federated learning to address the so-called straggler problem and communication bottlenecks of massive-sc ale training[26,28]. System Adaptively and Monitoring: Feedback loops, which are driven by monitoring, are a research field being actively pursued in the context of self-adaptive systems[27,28]. Monitoring system measures such as queue time and detection latency allow the system to dynamically optimize resources, which is required to ensure a low false positive rate and high resource utilization in dynamic network conditions[29]. Finally, the CA-AI design incorporates the well-known concepts of deep temporal learning. With the latest state-of-the-art architectural designs of concurrent, distributed, and real-time systems research, to make concurrency a first-class design concept to produce production-grade performance in demanding AI applications.

Recent advances in distributed and explainable IDS

There have been recent studies on intrusion detection that investigate complementary lines of research to make the system more robust and scalable to deploy. Hybrid Cluster Based Classification Model for Anomaly Detection in Unbalanced IoT Networks solves class imbalance issues by enhancing the accuracy of the anomaly detection in heterogeneous IoT traffic using clustering-based classification mechanisms[30]. Likewise, XAI FL-IDS combines

Federated Learning with explainability based on SHAP to enable distributed intrusion detection while enhancing transparency and interpretability of the models[31]. These approaches are geared towards data imbalance handling, distributed learning, and explainable decision making, but not towards concurrency management and scheduling efficiency, fault isolation, etc. in highly concurrent workloads. Thus, the proposed CA-AI framework complements these studies, with the focus being on concurrency-aware execution, actor-based isolation, asynchronous inference, and workflow-driven scheduling for scalable real-time IDS. Possible future research directions include embedding federated learning and explainable AI components into the CA-AI system.

Shortcomings of current concurrency solutions in AI systems

The conventional methods of managing concurrency in IDS systems are heavily thread-based in nature and they are based on the shared memory congruence, mutex locks, and centralized schedulers[32]. Although these methods can be used with sufficient performance at moderate workloads, they are highly scaled when implemented in large-scale SDN systems[33]. Empirical evidence has shown that lock contention and blocking I/O operations add nonlinear latency growth and decreased throughput at high concurrency[34]. Recent developments in distributed stream-processing systems, including Apache Spark streaming and Apache Flink offer data-parallelization, but they focus on micro-batches of execution and centrally coordinated execution[35,36]. Windowing, checkpoint synchronization and global state management add overhead in terms of latency that makes them less applicable to fine-grained, per-flow intrusion detection settings. Whereas the Actor Model has been a well-known and well-understood paradigm of scalable and fault-tolerant distributed systems, its synergy with deep temporal learning networks is largely understudied. Current systems of actors do not generally consider tightly integrating scheduling policy with AI inference needs.

Problems with LSTM-temporal models

LSTM networks are useful in sequential dependencies modeling of network traffic data. Nonetheless, parallel computation is limited by their sequential order of computation. This puts a conflict between the fidelity of temporal modeling and real time responsiveness in high-throughput environments. Furthermore, standardized LSTM training pipelines usually use synchronous gradient updates, which can be affected by the straggler effect and communication bottlenecks in the distributed environment. These drawbacks decrease convergence performance and waste more time in training. Even model deployment updates can interfere with the availability of inference unless asynchronous model versioning systems are used. Transformer-based temporal models are new solutions that are capable of parallel attention-based computation. However, their complexity in compute and memory space presents both novel scheduling and resource allocation issues in the presence of high concurrency.

Scientific problems not resolved

According to the reviewed literature, one can distinguish the following unsolved issues: Absence of co-design of concurrency architecture and deep learning models. Lack of risk-conscious AI-based scheduling solutions. Poorly supported fine-grained per-flow fault isolation. Poor feedback-based runtime optimisation. Loss of scalability when under bursty traffic.

The differences between traditional actor-based systems, LSTM based intrusion detection systems, workflow-based scheduling systems, and the proposed CA-AI system are summarized in Table 1. While the existing actor systems mainly deal with concurrency management and fault isolation, the LSTM based IDS models emphasize on temporal pattern learning and classification accuracy. The primary issues of workflow scheduling frameworks are resource allocation and ordering of executions in the face of dynamic workloads. The proposed CA-AI approach, on the other hand, combines all aforementioned aspects of concurrency management, temporal learning, risk-aware scheduling, asynchronous inference execution, monitoring-driven adaptation and unified

Table 1 Capability-based comparison of existing actor systems, LSTM-based IDS frameworks, workflow scheduling approaches, and the proposed CA-AI architecture.

Capability	Actor systems	LSTM IDS	Workflow scheduling	CA-AI
Concurrency management	✓	✗	Partial	✓
Temporal learning	✗	✓	✗	✓
Risk-aware scheduling	`✗	✗	Partial	✓
Async inference	✗	✗	✗	✓
Monitoring feedback	✗	✗	✗	✓
Unified co-design	✗	✗	✗	✓

architectural co-design into a common framework. This comparison shows that CA-AI is not just about adding a new actor model, a new learning algorithm, or a new scheduling mechanism, but aims for coordinated, systematic co-design and integration of these components to create scalable, low-latency and resilient pattern recognition in highly concurrent setting.

System architecture overview

The proposed CA-AI system integrates five tightly coupled layers:

1. Actor-based concurrency management
2. Workflow-oriented scheduling platform
3. Deep LSTM based learning for temporal pattern recognition
4. Agent-based parallel feature extraction
5. Monitoring and security feedback loops

Actor-based concurrency management—Describes how the actor model enables non-blocking communication, fault isolation, and scalable parallel task execution.

Workflow-oriented scheduling platform—Explains the load-aware scheduling mechanism that dynamically allocates computational tasks to optimize throughput and reduce latency.

Deep LSTM-based learning for temporal pattern recognition—Details the temporal modeling approach used for sequence learning and intrusion detection.

Agent-based parallel feature extraction—Presents the distributed feature computation strategy that accelerates preprocessing and imindicates concurrency efficiency.

Monitoring and security feedback loops—Describes the adaptive performance monitoring mechanism that updates the environment and maintains system stability and reliability.

Concurrency-aware coordination between actor layer and stacked LSTM

The stacked LSTM and actor-based concurrency layer are loosely coupled and functionally decoupled in the proposed CA-AI framework. The LSTM achieves temporal feature learning, and intrusion classification and concurrency management are executed through the actor system, which manages concurrency to provide scalability and predictable execution performance. The internal calculations of LSTM are not dependent on concurrency signals in order to maintain the stability of training. Awareness of concurrency is obtained by dynamically scheduling the LSTM inference pipeline, with per-flow actors synchronously delivering feature sequences and a load-aware scheduler dynamically controlling inference calls, batching, and flow-to-actor allocation using real-time feedback, including queue length and processing latency. The feedback-based coordination can allow backpressure to be successfully controlled and detect intrusions with low latency at high concurrency without changing the learning dynamics of the LSTM.

The design suggests the processing of every flow in the network separately and uses scalable and fault-tolerant processing without bottlenecks in shared state. Figure 1 represents a five-layered, concurrent system that is

constructed around the Actor Model of scalable pattern recognition. Users/Network Traffic first encounters the Data Ingestion Layer, where the Flow Manager Actors spread out data into parallel Flow Actors that process each data input stream and provide reliability. The central concurrency management layer is built on the Actor Model and is used to implement synchronization (lock-free, atomic operations) and load-sensitive, priority-based scheduling. The data is then processed simultaneously on one hand in a Feature Engineer layer (streaming features) and on the other in the DECISION & response layer (check-pointing and fault isolation handling). The learning + pattern recognition core utilizes an AI Model Actor to perform asynchronous deep learning (e.g., Transformer/Deep Model) to categorize patterns as Attack or Normal to initiate such responses as blocking IP addresses, rate limiting, or rerouting traffic. Lastly, a monitoring & security layer constantly follows the performance measures

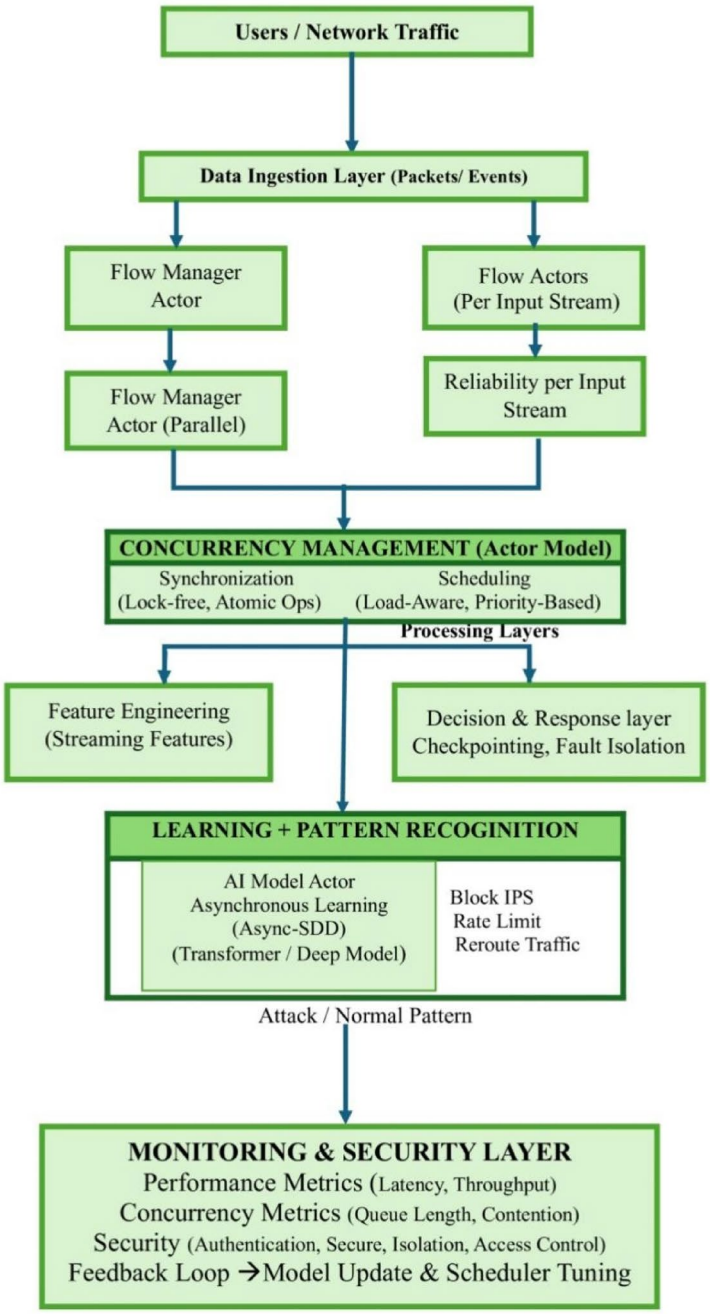


Fig. 1 Proposed five-layer CA-AI architecture for SDN-oriented intrusion detection

(latency, throughput), concurrency measures (queue length, contention), and security measures, and it is a feedback loop that optimally and dynamically adjusts AI model updates and the scheduler, thus providing an adaptive and resilient real-time operation. This formulation is based on the suggested CA-AI system design to give a formal representation of flow-level temporal and protocol-specific properties that are used to concurrently recognize patterns.

The Fig. 2 describes concurrency-sensitive artificial intelligence architecture for scalable pattern recognition in current software systems with an accent on effective management of large data volumes with high rates of real-time data streams. The first step taken to characterize incoming traffic is by using flow-level parameters such as the average packet rate, the sliding window size, and the number of extracted features, which guarantee consistency of temporal analysis. An agent-based parallel feature extraction operating system is then used to extract features with low execution latency, where statistical, temporal, and protocol-specific agents are used to extract comprehensive flow representations. These characteristics are computed by a workflow-based scheduling framework that ranks inference requests by risk scores of the AI model and rigid flow-time deadlines, which allows making decisions in time with a large number of concurrent requests. A feedback and monitoring system

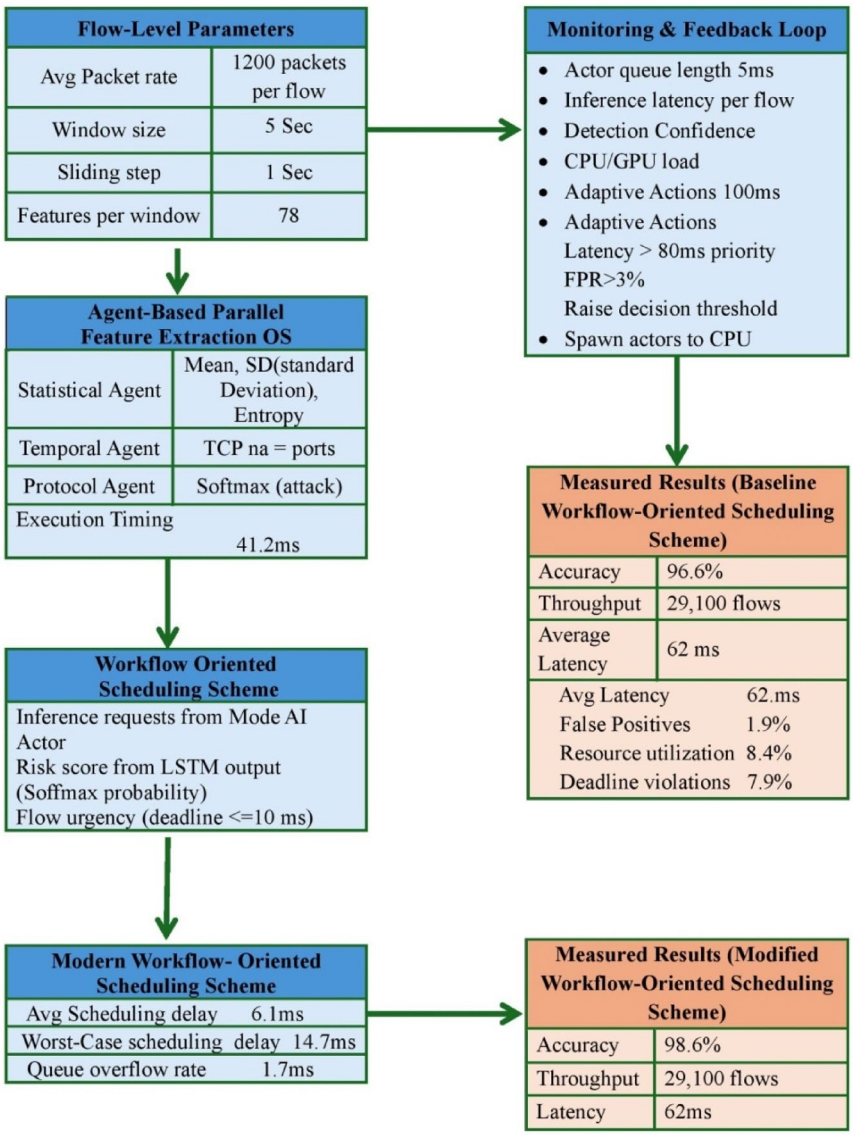


Fig. 2 Workflow scheduling and concurrency coordination mechanism

constantly monitors system metrics, including the length of actor queues, inference latency, detection confidence, and CPU/GPU usage, and adjusts system behaviour, including decision thresholds or spawning more actors to handle load. The framework compares of two settings of the baseline workflow-oriented scheduling scheme and a modified and contemporary scheduling scheme. Although the baseline determines the reference performance with regard to accuracy, throughput, latency, and resource utilization, the modified scheduler has a better accuracy and a more efficient scheduling delay at steady throughput and latency. Broadly, the number underscores the relevance of achieving scale, responsiveness, and reliability of AI-based pattern recognition systems through the incorporation of actor-based concurrency, adaptive scheduling, and feedback-driven control.

Flow-level temporal feature vector

$$X_f(t) = \{r_f(t), \Delta t_f(t), w_f, \phi_1, \phi_2, \ldots, \phi_c\} \quad (1)$$

where:

- $r_f(t)$ denotes the packet arrival rate of flow f at time t
- $\Delta t_f(t)$ represents the inter-arrival time of packets
- w_f is the sliding window size
- $\phi_1 \ldots \phi_c$ are the extracted statistical, temporal, and protocol-specific features

Equation (1) shows how flow-level temporal features are constructed for pattern recognition in network traffic.

The sliding window size in the experimental implementation w_f specifies the time frame in which flow-level features are calculated. In particular, packet arrival rate. $r_f(t)$ And inter-arrival time $\Delta t_f(t)$ the aggregation of t is over a fixed length sliding window of the most recent. w_f Packets or time steps. This window is like a moving buffer which is constantly updated as new packets come in. The resultant order of length w_f is the input to LSTM model of recognition of temporal patterns. In our experiments, w_f was empirically tuned to 20 through validation analysis, to trade-off the richness of temporal features and the inference latency in real-time.

The system as a whole represents a systematic end-to-end workflow whereby the network traffic entering the system is initially cleaned by Flow Actors to ingest and isolate it. This information is then sent to the agent-based feature extraction layer where statistical, temporal, and protocol-specific features are calculated in parallel. These features are passed to the workflow-oriented scheduler that prioritizes work depending on risk scores, deadlines and load on the system. The planned activities call on an asynchronous LSTM-based inference to classify patterns and the output is then handled in the decision layer to take mitigation measures. Lastly, the monitoring and feedback layer is in constant watch of system and detection measures to dynamically tune scheduling policies and resource allocation. This synchronized communication makes sure that it operates with high concurrency, efficiently and with low latency and scalability.

IDS workflow within the CA-AI framework

The proposed CA-AI framework is a real-time IDS with five processing steps: in (1) network traffic is ingested from Flow Actors, in (2) it is processed by the agents for parallel feature extraction, in (3) it is scheduled and prioritized based on the workflows, in (4) it is classified as intrusions in the background using an LSTM model, and in (5) a decision and a mitigation response are generated. All network flows are fed to separate Flow Actors that remove temporal and protocol-specific features and then send inference queries to the LSTM Model Actor. The detected data are then passed on to the response layer where they can be used to trigger mitigation measures like the blocking of traffic, limiting rates, or alerts. Such architecture provides scalable per-flow analysis and low latency and fault isolation with high concurrent workloads.

The obtained features are packet arrival rate, inter-arrival time, sliding-window statistics, protocol attributes, flow duration and temporal behavioral indicators. All these features combine to constitute the flow-level temporal feature vector as input to the stacked LSTM classifier.

Extracted features are packet arrival rate, inter-arrival time, sliding-window statistics, protocol attributes, flow duration and temporal behavioral indicators. All these features combine to create the flow-level temporal feature vector as input to a stacked LSTM classifier.

Actor-based concurrency management for per-flow isolation

The Actor Model is used to handle concurrency, whereby every incoming network flow is handled by a specific Flow Actor that both encapsulates the computation and the state. These actors are one-way communicators with no two-way communication. This design offers good per-flow isolation, where failures, overloads, or delays in one flow do not extend to other flows. The system provides scalability to tens of thousands of flows running concurrently by avoiding locks, and contention can be reduced greatly, leading to lock-free execution. Additionally, fault tolerance is an improved factor since failures are localized to the individual actors and do not impair the entire system. The Actor Model, as shown in Fig. 2, is an effective model in dealing with complex concurrent workflows and state transitions and still maintains consistency without using shared memory or locking mechanisms. The processing time decomposition is formulated on the basis of the per-flow actor execution model in CA-AI architecture, which quantifies the overall latency through the ingestion stage, feature extraction stage, inference, and decision stage. Equation (2) explains the decomposition of total per-flow processing latency into major execution stages.

Per-Flow Processing Time Decomposition

$$T_{\text{flow}} = T_{\text{ingest}} + T_{\text{feat}} + T_{\text{infer}} + T_{\text{decision}} \tag{2}$$

where:

- T_{ingest} denotes the data ingestion time
- T_{feat} represents the parallel feature extraction time
- T_{infer} is the asynchronous model inference time
- T_{decision} corresponds to the response and mitigation time

The Fig. 3 depicts an actor based e-commerce checkout process, which includes the interaction between ShoppingCart, Payment, and Shipping actors asynchronously. The results indicate to be per-actor state isolating, have concurrent processing, and fault-safe message passing. The workflow is an illustration of how complex tasks may be performed both in parallel without shared memory or blocking as in per-flow processing of CA-AI. This shows a simplified e-commerce checkout process as was implemented following the actor model. Firstly, the user input of checkout information is sent to a system controller application, which in turn sends an AddItems message to the ShoppingCart actor asynchronously, and the request is stored in the mailbox of the actor. The ShoppingCart actor then receives the handleCheckout (CheckoutInfo) method sequentially so that only one message is received at a time. In order to finalize the checkout, it simultaneously requests information that supports the checkout by sending asynchronous asks messages to the PaymentActor and ShippingActor to obtain payment and shipping information, respectively. These actors perform autonomously and concurrently, calculate the information needed, and send their responses back to the ShoppingCart actor. When this is done, the PaymentActor and ShippingActor send tell messages to the CustomerActor with the payment and shipping receipts; the messages are sent in a fire-and-forget manner without any anticipated reply. In general, this design puts state into every actor and limits the adaptation of state to sequential message processing, thus guaranteeing thread safety, concurrency, and high fault isolation throughout the system.

One of the architectural benefits of the proposed CA-AI framework is that it supports fine-grained (per-flow) fault isolation. Every network flow is wrapped in its own independent Flow Actor that has its execution state and processes messages in a sequence. This high encapsulation is used to ensure that failures like processing delays, resource exhaustion or runtime exceptions in one actor do not spread to other actors. In contrast to thread-based

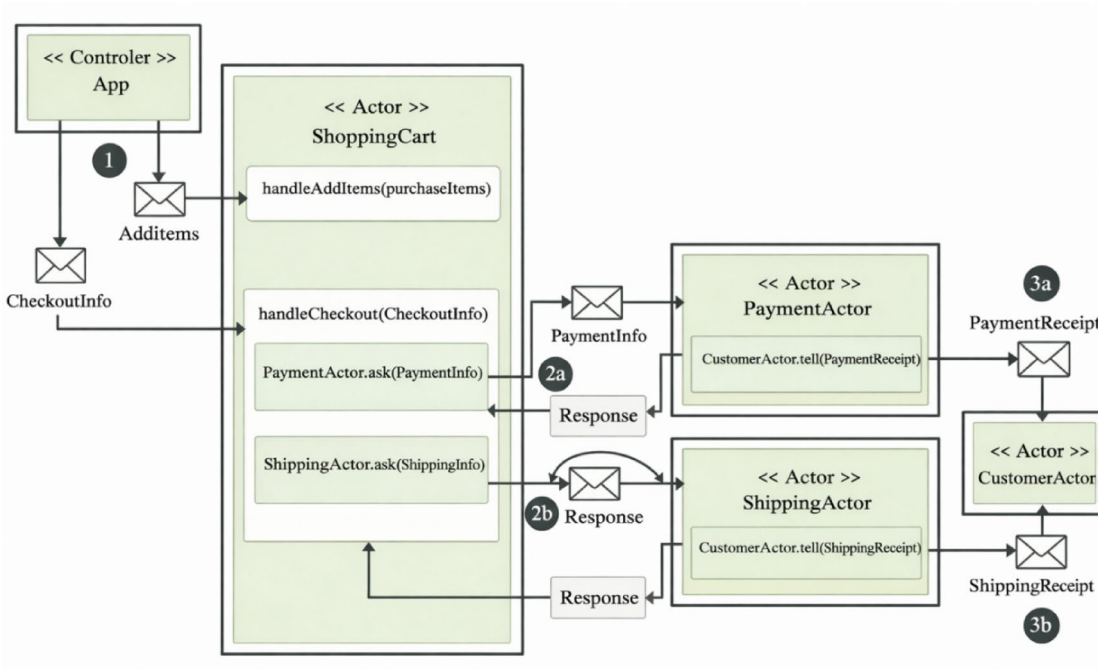


Fig. 3 Actor-based concurrent E-commerce checkout workflow

models, shared memory and synchronization primitives may cause cascading failures, the actor-based model does not have any shared state and instead uses asynchronous message passing to provide isolation. Moreover, fault recovery is localized on the level of actors. Flow Actor failure can be restarted without affecting other actors or the system workflow. This design has a great impact in that it minimizes the radius of failure and adopts quick recovery hence high availability of the system. This fine-grained fault isolation mechanism is quantitatively proven to be effective in "Reliability" section where it exhibits better flow isolation rate, recovery time and system availability.

Input:
Flow f
Compute Risk Score R_f
Retrieve Deadline D_f
Measure Queue Length Q_f
Compute Priority:
$Pf = \alpha R_f + \beta(1/D_f) + \gamma Q_f$
Sort Flows by Priority
If Actor Available
Assign Flow to Actor
Else
Spawn Additional Actor
End If
Trigger Async Inference
Update Monitoring Metrics

Algorithm 1 Risk-aware workflow scheduling

CA-AI scheduling algorithm

The proposed CA-AI framework has a risk-aware scheduling strategy, which is detailed in Algorithm 1. The scheduler calculates a priority score for every flow, based on the risk estimate, deadline constraints and runtime queue conditions generated by AI. The flows are then dynamically allocated to actors according to a priority and asynchronous inference is run while metrics are updated on a continuous basis. It provides scalable, low latency and adaptive processing in high concurrency situations.

Workflow oriented scheduling for load-aware execution

The intrusion detection process is represented by a workflow comprising several synchronized sub-tasks that are scheduled through a dynamic and load-sensitive scheduling object that takes into account the queue length of actors, resource availability, and flow risk scores. Reconfigurable workflow orchestration, in the form of agents, has been demonstrated to scale up and optimize cross-phase scheduling of complex software systems greatly[28]. In Fig. 4, the workflow-oriented scheduling platform is made to execute high performance, specifically to edge server infrastructures. It is initiated by the users entering an application request to the User Service, where the

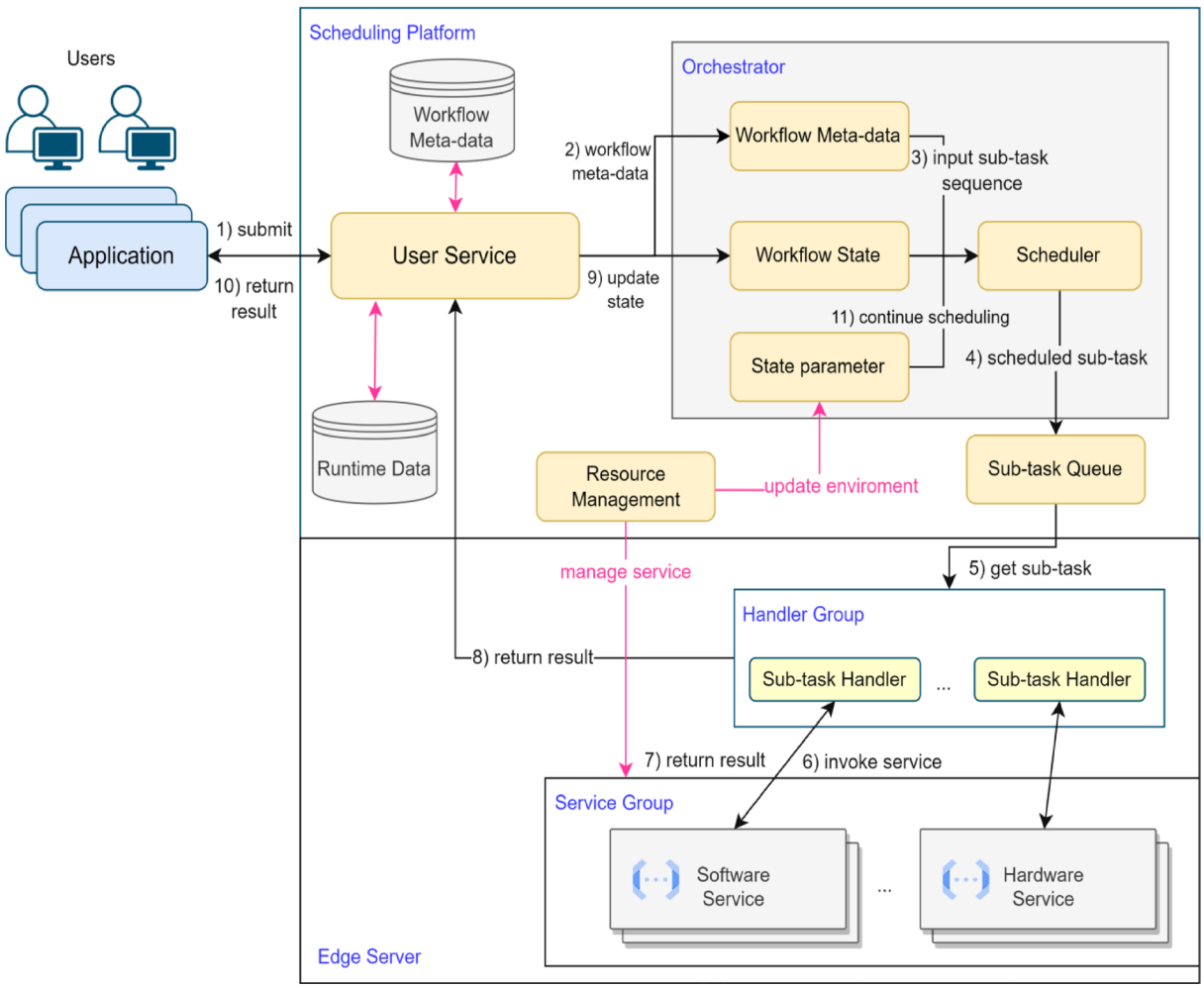


Fig. 4 Workflow-oriented scheduling platform architecture showing workflow meta-data driven orchestration and scheduling.

metadata of the workflow and the runtime data are retrieved and sent to the Orchestrator by the metadata. With this metadata, the Orchestrator establishes the sequence of sub-task executions and is used to update the Workflow State, which is consulted by the Scheduler to establish and schedule the next sub-task. The chosen sub-task is discovered in a sub-task queue; a sub-task handler in the handler group pulls it and executes it by calling the correct software or hardware service in the service group. When an execution is done, the results are sent back through the Handler Group to the Orchestrator, which will update the Workflow State and signal the User Service. In this process, the Resource Management part constantly monitors and updates the execution environment using state parameters of the Orchestrator to support efficient and adaptive scheduling. This architecture provides congestion avoidance, end-to-end latency reduction, and high-risk traffic prioritization, which will allow effective and responsive intrusion detection.

The workflow meta-data in the proposed framework is defined as the formal model of the intrusion detection workflow, which is the decomposition of tasks, dependencies among tasks, resource allocation, and scheduling constraints. Precisely it determines the overall detection process as broken down into sub-tasks (e.g., feature extraction, inference, and decision-making), sequence of execution between the sub-tasks and the constraints in the form of deadlines, priorities and resource requirements. Moreover, the flow level attributes include AI-based risk scores and time-based properties, which are included in workflow meta-data and utilized to make decisions regarding dynamic agenda. The meta-data of the workflow is stored in the scheduling platform and can be accessed by the Orchestrator to assemble the execution plan of sub-tasks. According to this meta-data, the Orchestrator will create an ordered sub-task sequence, which is passed to the Scheduler. The Scheduler will use workflow meta-data as well as real time state parameters to dynamically allocate tasks to the Sub-task Queue and the corresponding Handler Group to achieve efficient and adaptive execution.

The comparison of the scheduling efficiency of the traditional thread-based IDS and the proposed CA-AI IDS is shown in Fig. 5 and Table 2. This also shows the workflow meta-data component defines task structure, dependencies, and scheduling constraints used by the orchestrator. The findings indicate that CA-AI IDS will save a huge amount of scheduling overhead, previously at 12.8 ms down to 6.1 ms, because it has message-passing, lock-free coordination. Better distribution of workloads reduces the load imbalance ratio from 0.37 to 0.14, which imindicates the utilization of processors. In addition, CA-AI IDS gives precedence to the critical traffic, which is getting a greater latency gain on high-risk flows (48% over 21%), and dynamically adjusts scheduling decisions to achieve lower deadline miss ratios (18.6% to 7.9%). Altogether, these findings prove the high performance and reactiveness of the concurrency-aware scheduling method. The scheduling priority feature is presented as

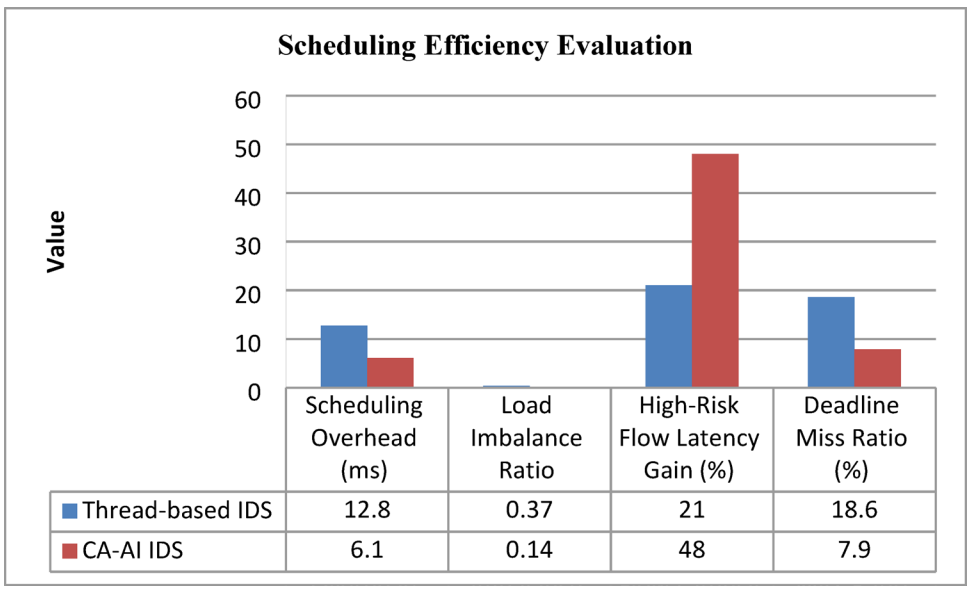


Fig. 5 Scheduling efficiency evaluation

Table 2 Scheduling efficiency evaluation.

Metric	Thread-based IDS	CA-AI IDS	Explanation of improvement
Scheduling overhead (ms)	12.8	6.1	Lock-free, message-passing coordination
Load imbalance ratio	0.37	0.14	Even workload distribution across processors
High-risk flow latency gain (%)	21%	48%	Critical flows prioritized
Deadline miss ratio (%)	18.6%	7.9%	Dynamic scheduling prevents overruns

a system-specific heuristic to combine AI-based risk and deadline constraints and runtime queue state into the workflow-based scheduler. Equation (3) shows how scheduling priority is computed using risk, deadline, and queue state.

Workflow-oriented load-aware scheduling priority

$$P_f = \alpha \cdot R_f + \beta \cdot \frac{1}{D_f} + \gamma \cdot Q_f \tag{3}$$

where:

- P_f denotes the scheduling priority of flow f
- R_f represents the AI-predicted risk score
- D_f is the deadline constraint of the flow
- Q_f indicates the current actor queue length
- α, β, γ are weighting coefficients controlling priority influence

Deep LSTM-based learning for temporal pattern recognition

Another key component of the proposed Concurrency-Aware Artificial Intelligence (CA-AI) framework is the deep temporal modeling. Although Long Short-Term Memory (LSTM) networks are highly developed in terms of sequence modeling, they are highly challenging in terms of their architecture, and execution in high-concurrency and real-time Software-Defined Network (SDN) environments. This section further elaborates on this discourse to define the methodological design as well as the scientific contribution of connecting LSTM to a concurrency-aware system.

LSTM architectural role in the CA-AI framework

The stacked LSTM model in the proposed architecture is not used as a centralized monolithic learner. Rather it is an actor-managed, asynchronous execution pipeline. Flow Actor sequentially receives temporal features and invokes inference service with the LSTM asynchronously, and stores a sequence of features in a sliding window. This design ensures:

- Isolation of sequence state Per-flow.
- Calls to non-blocking inference.
- Lack of shared-memory competition.
- predictable and bounded latency behavior under high concurrency.

The CA-AI framework differs with the traditional central implementation of IDS where all traffic is pooled and passed through one inference queue globally, instead of distributing the inference requests or requests to individual actors. This avoids inference bottlenecks and provides stable throughput growth under increasing concurrency.

Stacked LSTM model design

Recurrent neural networks have been shown to be very strong in security sensitive pattern embedding and pattern detection tasks in the past, which amplifies the appropriateness of these networks in temporal intrusion detection[29]. A deep stacked LSTM architecture is a network structure that learns temporal relationships of network traffic. Features are obtained on the basis of sliding time windows. Inference in multiple flow actors is done asynchronously, and asynchronous gradient aggregation allows the model to be continuously updated, as inference does not hold up.

Formulations of classification are the output phase of the proposed asynchronous LSTM inference pipeline in the CA-AI framework**.** Equation (4) explains how the LSTM model generates class probabilities for intrusion detection.

LSTM-Based Temporal Pattern Classification

$$\hat{y}_f = \text{Softmax}\,(W \cdot h_T + b) \tag{4}$$

where:

- h_T denotes the hidden state of the final LSTM layer
- W and b are the trainable weight matrix and bias vector, respectively
- $\hat{y}_f$ represents the predicted class probability vector (e.g., *Attack* / *Normal*)

Figure 6 shows a deep Stacked Long Short-Term Memory (LSTM) Architecture, which is temporal pattern recognition architecture frequently applied to sequence data such as time-series features in network traffic. The architecture will start with the input layer that is supplied with a sequence of n features, where c is the number of different types of features, and the input is a matrix of size n x c. This sequence is then entered into four consecutive LSTM layers (LSTM1 through LSTM4). Each of the four LSTM layers consists of 64 hidden units, whereby the sequence is processed at time steps t1 to t64, and the sequence information is propagated to the next layer in the stack. The result of the last LSTM layer (LSTM4) is a sequence of 64 1 vectors and flattened into a final dense layer. This thick layer converts the hidden state representations to the dimension to be used in the classification. Lastly, the SoftMax Output layer performs the SoftMax operation to generate a probability distribution across c classes, meaning that the model can predict the input sequence to fall in one of a variety of possible categories, e.g., various forms of network attacks.

Figure 7 and Table 3 shows the performance of the Async CA-AI Learning scheme shows a considerable performance gain over the classical model, which is the Centralized Learning scheme, mostly because it is designed to perform asynchronous and parallel operations. This concurrency-sensitive model inference time is reduced significantly by more than 50% (8.7 ms to 4.2 ms) with parallel, non-blocking inference by special flow actors. Moreover, the system speeds up the development cycle through reduced training convergence time of 52 to 38 epochs through distributed gradient aggregation. More importantly, the Model Update Downtime is practically removed, as it is reduced to less than 5 ms by use of hot-swapping model versions and suggests permanent availability of service. It is also an efficient way of utilizing available resources, as this distributed strategy shrinks the memory footprint by a quarter since there is less use of shared memory.

Concurrency-aware asynchronous inference pipeline

The main scientific value of this work is the conversion of a traditional LSTM synchronous inference model into an asynchronous pipeline that is concurrency-aware.

With traditional implementations:

Inference is blocking.

The use of batch scheduling creates delay.

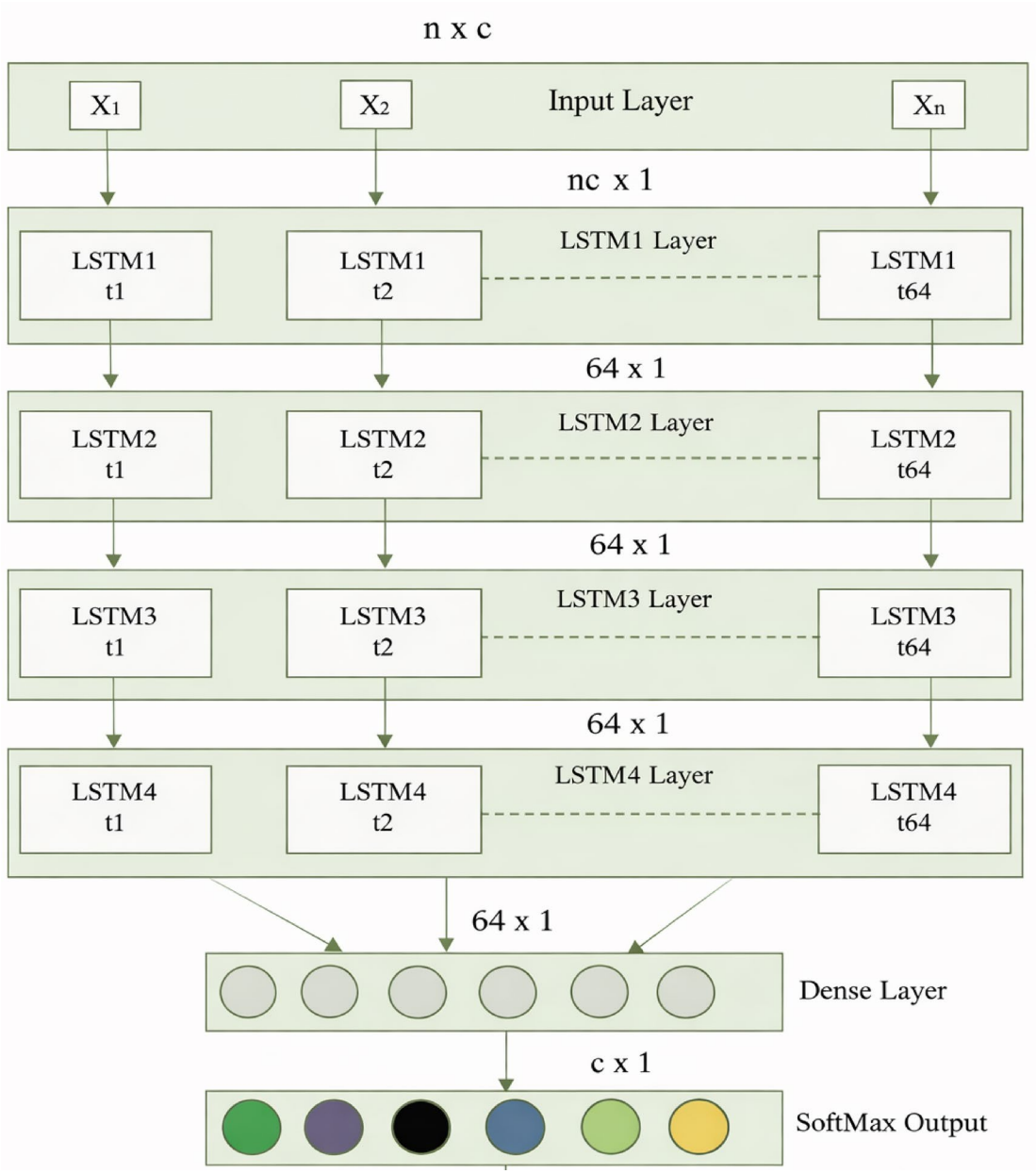


Fig. 6 Asynchronous actor-managed stacked LSTM architecture

The updates in models lead to globally widespread breaks.
The CA-AI framework, in its turn, brings about:

1. actor-mediated inference calls.

Flow Actor processes each request to the inference server independently without coordination on a global basis.

2. Non-blocking message passing.

The LSTM Model Actor handles the requests in an asynchronous manner which removes the lock contention.

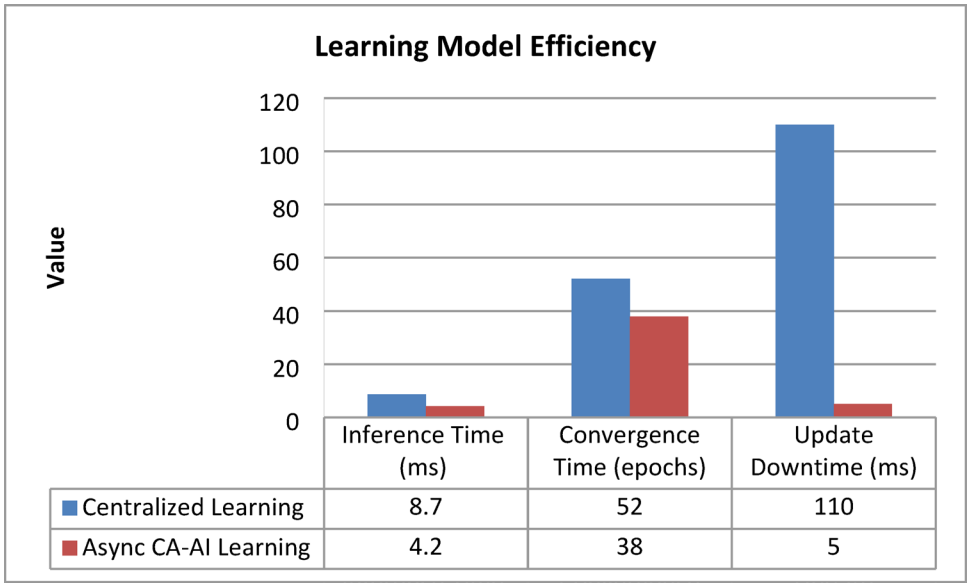


Fig. 7 Learning model efficiency comparison: centralized versus Async CA-AI

Table 3 Learning model efficiency.

Metric	Centralized learning	Async CA-AI learning	Explanation
Inference time (ms)	8.7	4.2	Parallel, non-blocking inference
Training convergence time (epochs)	52	38	Distributed gradient aggregation
Model update downtime (ms)	110	<5	Hot-swapping model versions
Memory footprint (MB)	960	720	Reduced shared memory usage

3. Dynamic batching based on runtime load.

   The workflow scheduler does throughput and latency balancing through queue-length feedback by adjusting batching.

4. Backpressure-aware scheduling

   When the size of actor mail boxes crosses thresholds, then scheduler priorities are dynamically adjusted in order to avoid congestion.
   This design reduces inference time of 4.2 ms instead of 8.7 ms (centralized) in the experimental assessment.

Distributed gradient aggregation and continuous learning

The CA-AI system training favors asynchronous gradient aggregation. Rather than synchronised updates that block inference:

- Local gradient contributions are added to Flow Actors.
- The asynchronous aggregation of gradients.
- Parameters of the model are updated without stopping inference services.

This prevents the classical straggler effect of distributed training and makes convergence take 38 instead of 52 epochs.

Moreover, hot-swapping mechanism is used to deploy model updates, in which:

- Another version of the model is loaded simultaneously.
- Active inference passes over.
- Downtime is reduced to < 5 ms.

This makes it always available which is important in mission-critical SDN environments.

Scientific contribution of the LSTM integration

The fact that this work does not suggest a new LSTM algorithm should be explained. Instead, it makes scientific contributions in:

- Adaptive concurrency Formal co-design of LSTM temporal modeling and actor-based concurrency.
- Conversion of synchronous LSTM execution to asynchronous actor-controlled pipeline.
- Risk-consciousness risk-conscious integration of risk-conscious scheduling.
- Installation of blanket model changes at a near-zero downtime. Experimental demonstration of predictable low-latency behavior low-concurrency (high: 100,000 flows) performance.

This combination of architecture allows the LSTM model to perform with high-throughput, real-time functionality, by breaking scalability and synchronization bottlenecks that are typically ignored in deep learning-enabled IDS studies.

Practical impact

The concurrency-sensitive LSTM execution model directly annotates the system improvements which are measurable:

- 2.1 × throughput increase
- Average latency reduction: 57.5% compared with the centralized IDS baseline (146 ms to 62 ms)
- 99.6% system availability
- 25% memory footprint reduction

These gains confirm that architectural design that is concurrency aware is as important as model accuracy in real time AI systems.

Agent-based parallel feature extraction

The extraction of features is split into independent feature agents that deal with statistical, temporal, and protocol-specific features. Agents operate concurrently and give their results asynchronously.

The Fig. 8 represents the Agent-Based Parallel Feature Extraction Model, which demonstrates a hierarchical and parallel method of the input data processing. The process is initiated by an input that is processed by the initiator and collector agent. It is this initiating agent that delegates certain tasks in parallel to a number of sub-agents, which are identified as Agent 1 through Agent n. These main agents (or agents such as Agent 1, which in turn breaks down tasks into Sub-agent 1.1 and Sub-agent 1.2) have their own dedicated model, knowledge, and tools to execute a given function such as computing a set of features. On completion of each agent, an intermediate result is sent to the initiator, the collector agent. Lastly is the process of the collector taking these disparate intermediate results, which involves the combination, comparison, and selection of the outputs that are applicable to create the aggregated result that will be used downstream. This distributed model minimizes processing latency and maximizes system throughput through the exploitation of parallel computation.

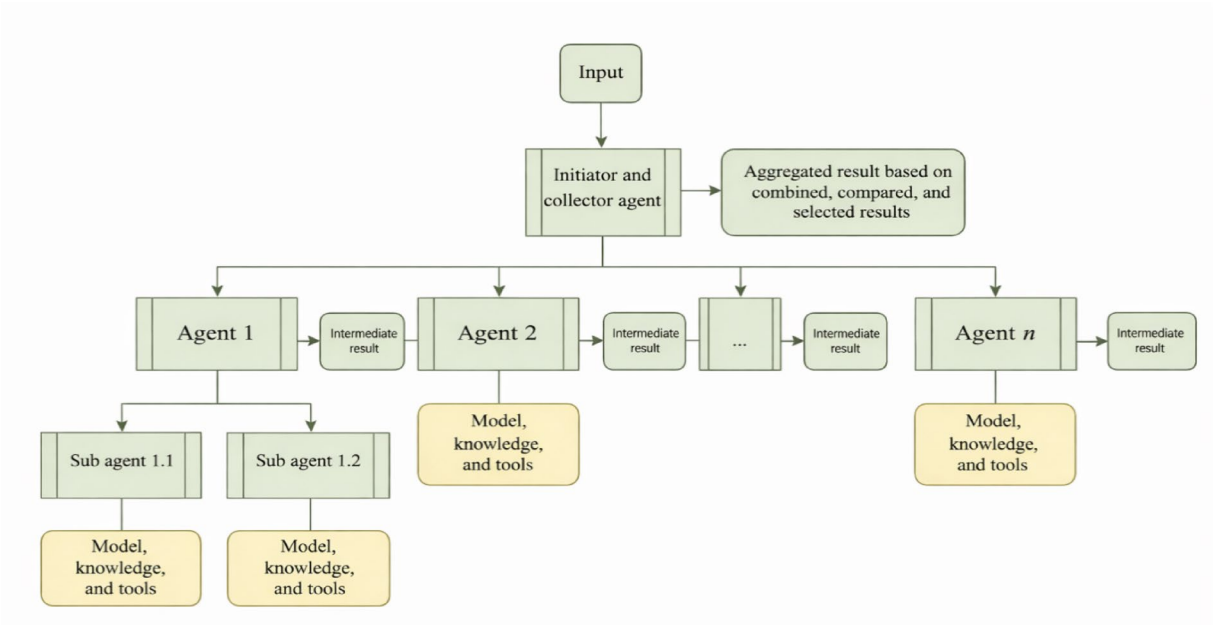


Fig. 8 Agent-based parallel feature extraction model architecture

Table 4 Feature extraction performance.

Metric	Sequential	Parallel agents	Improvement	Explanation
Feature extraction time (ms)	41.2	18.9	2.18×	Simultaneous computation
CPU utilization (%)	72	86	+14%	Better core utilization
Synchronization cost (ms)	–	2.1	–	Minimal cost via async messaging

Although individual Flow Actors perform individually to identify per-flow behavior, distributed attacks like DDoS and port scans are identified using a hierarchical aggregation approach. Flow Actor reports to the Collector Agent asynchronously the compact flow-level summaries, including anomaly confidence scores, burst intensity, and temporal markers of activity, which are sent via the Collector Agent. The Collector Agent combines these summaries of several flows through sliding time windows to recognize correlated patterns of attack that are shared by large numbers of independent flows.

Table 4 demonstrates that the Parallel Agents method provides a 2.18 times faster Feature Extraction Time (to 18.9 ms) than the Sequential one, as it allows computing things in parallel and harnessing 14 percent more CPU bandwidth. With such efficient asynchronous messaging, this gain is realized with just a slight synchronization cost of 2.1 ms.

Monitoring and security feedback loops

A monitoring layer monitors system metrics (actor mailbox sizes, detection latency, and confidence) to adapt dynamically scheduler priorities, learning rates, and resource allocation. Table 5 and Fig. 9 indicate that the monitoring and feedback loop was highly effective, reducing the average detection latency from 82 to 62 ms and the false positive rate from 3.4 to 1.9% through optimal resource allocation and adaptive thresholding. The efficiency of the systems was also improved by this on-going process, as the resource utilization increased from 69 to 84 percent by making 14 scheduler reconfigurations a minute in order to achieve the dynamic stability.

Table 5 Monitoring and adaptive optimization impact.

Metric	Before feedback	After feedback	Explanation
Avg detection latency (ms)	82	62	Optimized resource allocation
False positive rate (%)	3.4	1.9	Adaptive thresholds
Resource utilization (%)	69	84	Efficient workload distribution
Scheduler reconfigurations/min	–	14	Dynamic adjustments for stability

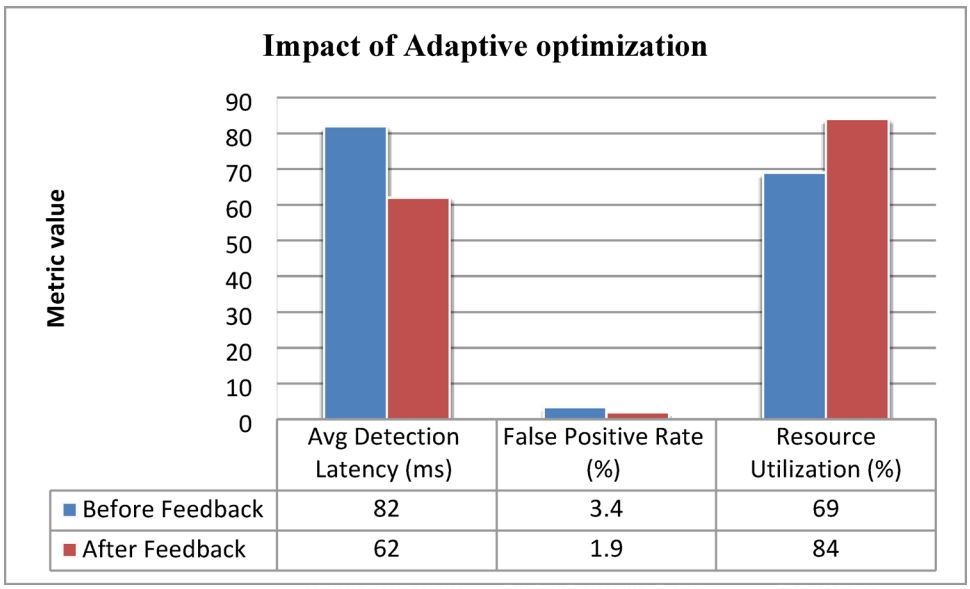


Fig. 9 Impact of adaptive optimization (monitoring and security feedback loops)

The feedback and monitoring layer is vital in the distributed attack detection since it ensures a global view based on aggregated actor-level measures. Sudden trends in the number of aberrant flows, coordinating timing trends, and the fan-out of source or destination are all indicators that are continually analyzed. In case of collective thresholds being surpassed, the system detects distributed attacks and dynamically changes scheduler priorities, mitigation policies, and response strategies and retains per-flow isolation.

Results and performance evaluation

Experimental setup and datasets

Well-established benchmark intrusion detection datasets are used in the evaluation to make it comprehensive and realistic. In particular, NSL-KDD is used to model classical intrusion cases, CIC-IDS2017 is used to model modern and more advanced attack cases, and UNSW-NB15 is used to model a variety of recent attacks. The proposed framework is evaluated by benchmark intrusion detection datasets and an SDN-inspired experimental environment that mimics flow-level traffic processing characteristics, in terms of scalability and resilience. This assessment is done with regard to the validation of concurrency management, scheduling efficiency and performance of intrusion detection under different levels of concurrent network flows, and different scales of network flows are simulated, i.e., 10,000 and 100,000. Each experiment was executed five times under identical conditions, and the reported results represent the average observed values.

The experiments were conducted on a workstation with the following detailed configuration: an Intel Core i7-12700 processor (12 cores, 20 threads, base frequency 2.1 GHz, turbo up to 4.9 GHz), 16 GB DDR4 RAM, and an NVIDIA RTX 3060 GPU with 12 GB GDDR6 memory. The system was running Ubuntu 20.04 (64-bit). The software environment included Python 3.9, TensorFlow 2.x and PyTorch (CUDA-enabled), with CUDA 11.x and

cuDNN support for GPU acceleration. All experiments were executed under consistent runtime conditions, and GPU acceleration was enabled for both training and inference phases to ensure fair and reproducible performance evaluation. The variation across runs was small and did not affect the overall ranking of competing methods.

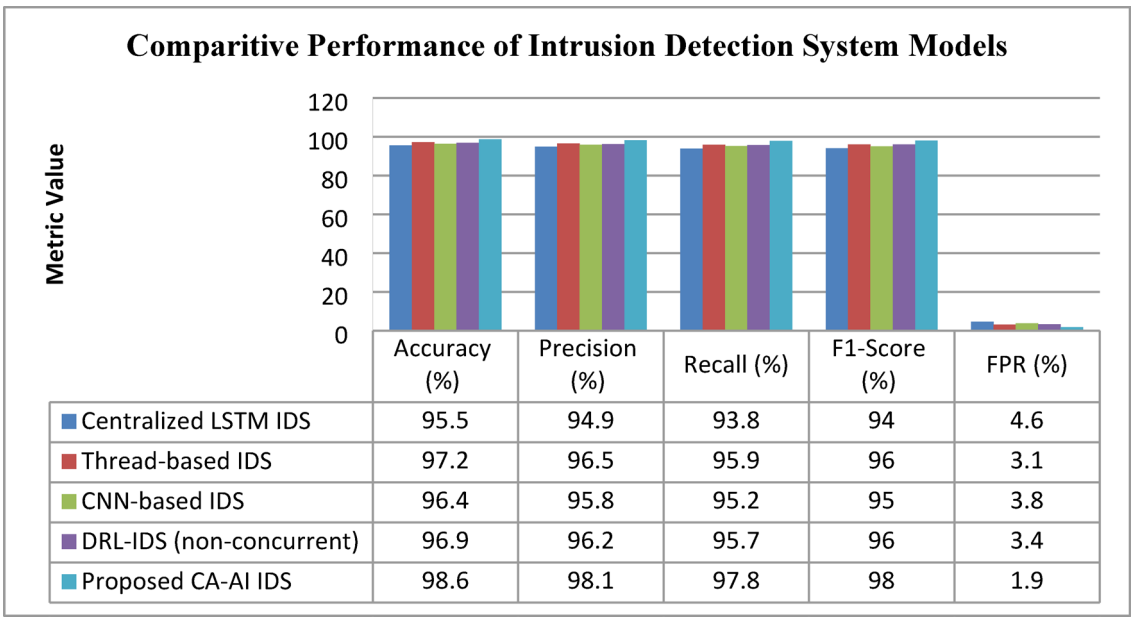
A fixed sliding window size is used in the process of temporal feature extraction. $w_f = 20$, that is the length of the sequence used as LSTM input. The flow is modelled as a series of 20 successive time steps, and is calculated as features of the flow, including the rate of arrival of packets and the inter-arrival time in the window. The size of the window was also selected based on empirical consideration to provide adequate temporal context despite low-latency inference.You can write it in a journal-style paragraph as follows:

The stacked LSTM model was configured with four LSTM layers, each containing 64 hidden units to capture temporal dependencies in network traffic sequences. Model training was performed using the Adam optimizer with a learning rate of 0.001 and a batch size of 128. The network was trained for 50 epochs, and a dropout rate of 0.2 was applied to reduce overfitting and improve generalization performance. Prior to training, dataset preprocessing was performed to ensure data quality and consistency. This process included normalization of numerical attributes, removal of duplicate records, encoding of categorical features into numerical representations, and construction of temporal input sequences using sliding window-based feature extraction. These preprocessing steps enabled effective temporal pattern learning and improved the robustness of the intrusion detection model across different datasets.

Detection accuracy

Figure 10 and Table 6 show that the proposed CA-AI IDS had a maximum detection accuracy of 98.6, and other models could not perform as well as this model. It also indicated the optimal combination of precision and recall, having an F1 score of 0.98. More importantly, the CA-AI model had a significant decrease in the False Positive Rate (FPR) to only 1.9%. This experiment validates the high performance of the concurrency-aware design in intrusion detection in real time.

Figure 11 and Table 7 indicate that the CA-AI IDS had the highest accuracy in all three datasets, with a maximum of 98.9 percent on NSL-KDD. The framework proved to be stronger than the centralized and thread-based



	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	FPR (%)
Centralized LSTM IDS	95.5	94.9	93.8	94	4.6
Thread-based IDS	97.2	96.5	95.9	96	3.1
CNN-based IDS	96.4	95.8	95.2	95	3.8
DRL-IDS (non-concurrent)	96.9	96.2	95.7	96	3.4
Proposed CA-AI IDS	98.6	98.1	97.8	98	1.9

Fig. 10 Comparative performance of intrusion detection system models

Table 6 CIC-IDS2017 detection performance.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score	FPR (%)
Centralized LSTM IDS	95.5	94.9	93.8	0.94	4.6
Thread-based IDS	97.2	96.5	95.9	0.96	3.1
CNN-based IDS	96.4	95.8	95.2	0.95	3.8
DRL-IDS (non-concurrent)	96.9	96.2	95.7	0.96	3.4
Proposed CA-AI IDS	98.6	98.1	97.8	0.98	1.9

Fig. 11 Dataset accuracy comparision

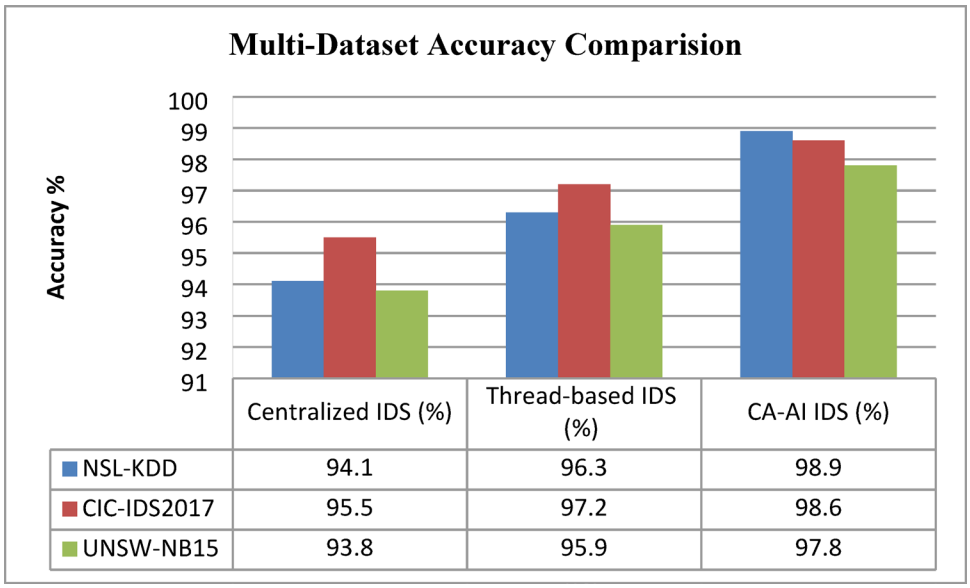


Table 7 Multi dataset accuracy comparison.

Dataset	Centralized IDS (%)	Thread-based IDS (%)	CA-AI IDS (%)
NSL-KDD	94.1	96.3	98.9
CIC-IDS2017	95.5	97.2	98.6
UNSW-NB15	93.8	95.9	97.8

IDSs, which were reportedly lower than 98%. This supports the fact that the concurrency-conscious architecture is very practical and dependable when it comes to recognizing patterns in varying network traffic settings.

Scalability

The throughput behavior of the considered intrusion detection architectures is presented in Fig. 12 and Table 8, where the number of parallel network flows is varied between 10,000 and 100,000. There is an observable reduction in throughput of the centralized IDS and thread-based IDS at high concurrency of about 50 and 40, respectively, with 100,000 flows. The shared-memory contention, thread synchronization overheads, and limited fault isolation are the most common factors that have led to this decline, and they are the performance bottlenecks that are dominant at scale.

Conversely, the suggested CA-AI IDS has much better scalability across the framework of the experiments conducted. With higher concurrency, CA-AI architecture stabilizes and even exceeds throughput with the range of 27,300 to 32,400 flows per second at 100,000 concurrent flows. Per-flow actor isolation, asynchronous message passing, and workflow-based and load-conscious scheduling (that enable this behavior) minimize contention and enable the system to dynamically respond to increasing workloads.

These findings suggest that an architectural design that is concurrency-conscious can achieve stable throughput growth under increasing concurrency even when operating in the given experimental conditions when using

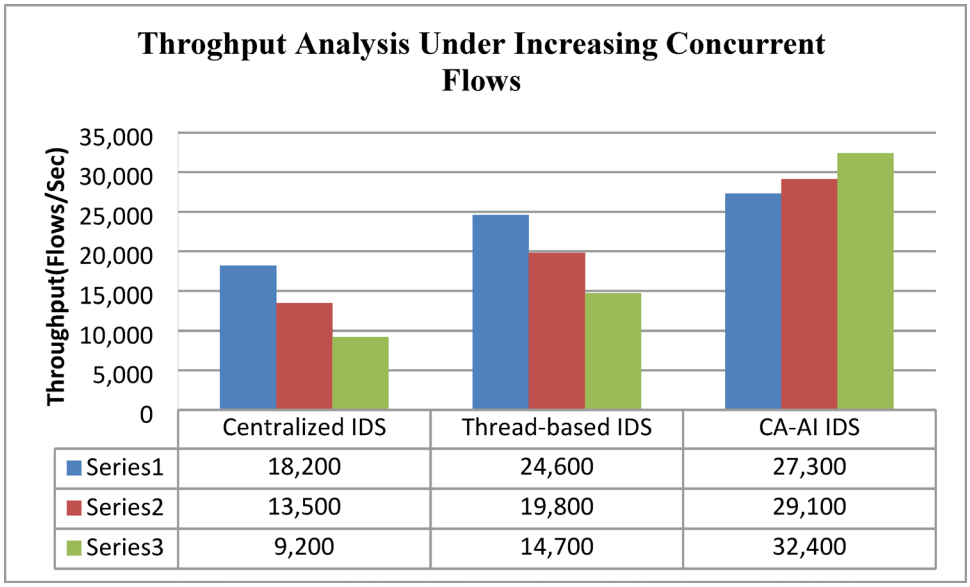


Fig. 12 Throughput analysis under increasing concurrent flows

Table 8 Throughput analysis.

Concurrent flows	Centralized IDS	Thread-based IDS	CA-AI IDS
10,000	18,200	24,600	27,300
50,000	13,500	19,800	29,100
100,000	9,200	14,700	32,400

high-volume and bursty network traffic. Nevertheless, the scalability traits observed are particular to the experimented SDN-based environment, datasets, and workload settings. Although the results are a solid indication of the efficiency of the CA-AI model when used in large-scale, real-time intrusion-detection scenarios, additional research on the use of the framework in larger deployment environments, including a heterogeneous hardware platform, a larger distributed controller, or different traffic models, is needed to help completely extrapolate the scaling capabilities of the proposed architecture.

In order to quantify the observed stable throughput growth under increasing concurrency, A scalability efficiency metric (SE) was defined as the ratio of throughput scaling to concurrency scaling:

$$SE = \frac{\left(\frac{T_2}{T_1}\right)}{\left(\frac{C_2}{C_1}\right)}$$

where:

- SE = Scalability efficiency
- T_1 = Initial throughput
- T_2 = Final throughput
- C_1 = Initial concurrency level
- C_2 = Final concurrency level

Experimental results of the proposed work s:

$$SE = \frac{\left(\frac{32400}{27300}\right)}{\left(\frac{100000}{10000}\right)} \quad SE = \frac{1.187}{10} \quad SE = 0.1187 \approx 0.119$$

The throughput of the proposed CA-AI framework in this case improved from 27,300 to 32,400 flows/s (1.19 × increase), while the latency of the system remained stable with the increase in the number of concurrent flows from 10,000 to 100,000 (10 × increase). The throughput of the CA-AI framework was not degraded at higher loads and showed positive concurrency scaling within the range of concurrency tested, whereas centralized and thread-based approaches suffered from throughput degradation as loads increased. The throughput of the CA-AI framework increased from 27,300 to 32,400 flows/s as concurrency increased from 10,000 to 100,000 flows. Thus, the observed behaviour is more appropriately characterized as stable throughput growth under increasing concurrency, without throughput collapse over the evaluated workload range.

The reported 2.1 × throughput improvement compares the proposed CA-AI framework with the centralized IDS baseline under identical workload conditions. In contrast, the 1.19 × throughput increase quantifies the throughput growth achieved by the proposed framework itself when the concurrency level increases from 10,000 to 100,000 concurrent flows. These two measurements represent different experimental comparisons and therefore should not be interpreted interchangeably. Table 9 shows the outcomes indicate that the CA-AI framework maintained the throughput efficiency while avoiding throughput collapse with 10 time's higher number of concurrent flows. This behavior demonstrates stable throughput growth without throughput collapse over the evaluated workload range.

It boosted the throughput from 27,300 flows/s and 10,000 concurrent flows to 32,400 flows/s and 100,000 concurrent flows. The CA-AI framework increased throughput from 27,300 flows/s at 10,000 concurrent flows to 32,400 flows/s at 100,000 concurrent flows. Unlike the centralized and thread-based systems, which exhibited throughput degradation as concurrency increased, CA-AI maintained stable throughput growth over the evaluated workload range. As such, the behavior observed can be described as stable throughput growth under increasing concurrency over the range of workloads tested.

Latency

Figure 13 and Table 10 show that the CA-AI IDS achieved an average latency of 62 ms, compared with 146 ms for the centralized IDS baseline and 108 ms for the thread-based IDS. Thus, the CA-AI IDS achieved an approximately 57.5% reduction in average latency relative to the centralized IDS baseline. The 95th and 99th percentile latencies were 118 ms and 187 ms, respectively.

The reported 43% latency reduction refers to the reduction achieved under the primary experimental comparison presented in the architectural evaluation. The approximately 57.5% reduction obtained from Table 11 represents the average latency improvement of the proposed CA-AI framework relative to the centralized IDS baseline (146 ms to 62 ms). Since these values originate from different evaluation settings, they should not be interpreted as identical performance metrics. Table 12 shows the other latency stability statistics were also analysed to further evaluate the predictability of latency. The standard deviation as well as coefficient of variance of the latency was 14.8 ms and 23.9% respectively, suggesting low variation around the mean latency. The peak latency observed was 224 ms during peak load conditions. From these findings, it can be concluded that the proposed architecture has bounded and predictable latency behavior in the tested environment. From these findings,

Table 9 Scalability characterization of CA-AI.

Metric	Value	Meaning
Concurrency Increase	**10 ×**	Concurrent flows increased from 10,000 to 100,000 (100,000/10,000 = 10)
Throughput Increase	**1.19 ×**	Throughput increased from 27,300 to 32,400 flows/s (32,400/27,300 = 1.187 ≈ 1.19)
Throughput Retention	**118.7%**	Throughput at highest load is 118.7% of throughput at lowest load
Scalability Behaviour	Stable throughput growth under increasing concurrency	No throughput degradation under increasing concurrency

Significant values are in bold.

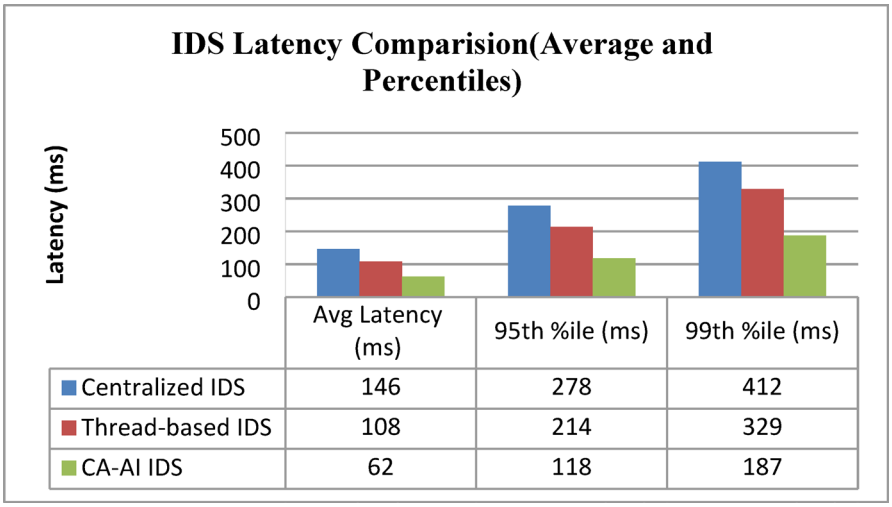


Fig. 13 End-to-end latency comparison

Table 10 End-to-end latency.

Model	Avg latency (ms)	95th %ile (ms)	99th %ile (ms)
Centralized IDS	146	278	412
Thread-based IDS	108	214	329
CA-AI IDS	62	118	187

Table 11 Additional latency stability statistics.

Metric	CA-AI IDS
Average latency (ms)	62
Standard deviation (ms)	14.8
Coefficient of variation (%)	23.9
Maximum observed latency (ms)	224
95th Percentile (ms)	118
99th Percentile (ms)	187

Table 12 Fault isolation.

Metric	Thread-based IDS (%)	CA-AI IDS (%)	Explanation
Flow isolation rate (%)	73	96	Per-flow isolation via Actors
Failure impact radius (%)	27	3.8	Faults localized
Recovery time (ms)	420	95	Rapid actor restart
System availability (%)	97.4	99.6	Enhanced resilience

it can be concluded that the proposed architecture exhibits predictable and bounded latency behavior within the evaluated experimental environment.

The graph shows that the average latency and tail latency measurements are closely aligned and performance remains stable at higher concurrency. The results indicate predictable latency behavior under the evaluated workloads, with bounded latency characteristics within the experimental environment.

Reliability

As indicated in Fig. 14 and Table 12, based on the metrics of Fault Isolation, CA-AI IDS has a far greater resilience than thread-based IDS. The Flow Isolation Rate of 96% (vs. 73%) of the CA-AI model was obtained through its use of a per-flow Actor model, effectively limiting faults to a 3.8% Failure Impact Radius (vs. 27%). Moreover, it utilized the rapid Actor restart, which enabled an extremely rapid recovery (reducing the recovery

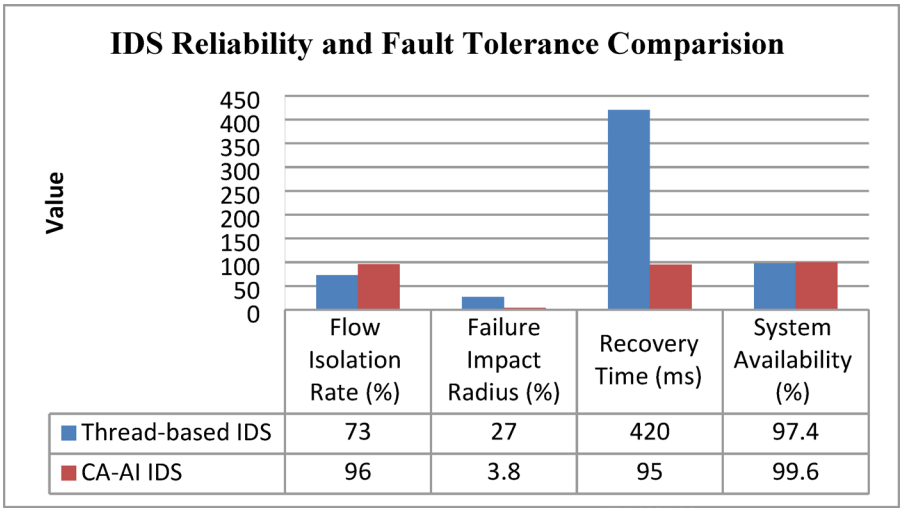


Fig. 14 Fault isolation metrics comparison

Table 13 Comparative analysis of integration-oriented AI frameworks and the proposed CA-AI architecture.

Aspect	Typical integration frameworks	Proposed CA-AI
Integration style	Layered / pipeline-based	Co-designed, concurrency-first
Concurrency handling	Implicit, thread-based	Actor-based per-flow isolation
Scheduling	Static or batch-oriented	Dynamic, risk-aware workflow scheduling
Learning execution	Centralized or synchronous	Asynchronous, actor-managed
Fault isolation	Coarse-grained	Fine-grained (per flow)
System suggests	Best-effort	Scalable, low-latency, resilient

time to a minimum of 95 ms instead of 420 ms), and the system availability was much higher, at 99.6%. These reliability improvements are directly attributed to the fine-grained, per-flow fault isolation enabled by the actor-based architecture.

The Table 13 presents the main architectural distinctions between default integration-oriented artificial intelligence systems and the suggested CA-AI system. It provides evidence that CA-AI is the only system that uses a concurrency-first, co-designed integration model in order to attain scalable, low-latency, and resilient performance in high-concurrency workloads.

Table 14 indicates that the proposed CA-AI framework is the only one that has incorporated concurrency-aware design, asynchronous execution, and risk-driven scheduling, which can be better utilized in real-time and high-concurrency environments than current methods.

Unlike representative of the literature IDS architectures, the proposed CA-AI framework has a novel combination of intrusion detection accuracy, concurrency-aware execution, workflow-driven scheduling, and throughput-oriented evaluation as shown in Table 15. Current research focuses on classification performance, while CA-AI takes into account scalability, latency, fault isolation, and real-time execution requirements.

Although a full ablation study is outside the scope of the present work, the contribution of each subsystem can be inferred from the corresponding performance metrics reported in “Workflow oriented scheduling for load-aware execution"-"Monitoring and security feedback loops” section. Table 16 shows the main function of each constituent part of the proposed CA-AI architecture. The advantages of actor-based concurrency include better fault isolation, better scalability, better workflow scheduling to improve execution efficiency by dynamically scheduling workflows based on their priority, better asynchronous inference to reduce processing latency, and better monitoring feedback to allow runtime adaptation and system stability. These elements combine for the performance enhancements seen in the experimental evaluation.

Table 14 Comparative analysis of CA-AI with recent intrusion detection approaches.

Aspect	Centralized LSTM IDS	Thread-based IDS	Stream processing (Spark/Flink)	DRL-based IDS	Proposed CA-AI
Concurrency handling	Limited (centralized)	Thread-based, contention-prone	Data-parallel, batch/micro-batch	Adaptive but not concurrency-aware	Actor-based, per-flow isolation
Execution model	Synchronous	Multi-threaded	Micro-batch / window-based	Policy-driven	Fully asynchronous
Latency	High	Moderate	Moderate to High	Moderate	Low (real-time)
Throughput	Limited	Moderate	High (batch-oriented)	Moderate	High (2.1 × improvement)
Fault isolation	Low	Medium	Medium	Medium	High (per-flow isolation)
Scheduling strategy	Static	OS-level	Centralized scheduler	Learning-based	Risk-aware workflow scheduling
Scalability	Limited	Moderate	High	Moderate	Stable throughput growth under increasing concurrency
Suitability for real-time IDS	Limited	Moderate	Limited (due to batching)	Moderate	Highly suitable

Table 15 Comparison with recent IDS studies.

Study	Dataset	Accuracy (%)	Concurrency-aware	Throughput Analysis
Centralized LSTM IDS	CIC-IDS2017	95.5	No	No
Thread-based IDS	CIC-IDS2017	97.2	Partial	Limited
DRL-IDS	CIC-IDS2017	96.9	No	No
Proposed CA-AI	CIC-IDS2017	98.6	Yes	Yes

Table 16 Component contribution analysis.

Component	Primary benefit
Actor-based concurrency	Fault isolation
Workflow scheduling	Reduced deadline misses
Async inference	Lower inference latency
Monitoring feedback	Adaptive optimization

Discussion

The experimental results demonstrate that incorporating concurrency-aware architectural principles into AI-driven intrusion detection substantially improves system performance under high-concurrency workloads. The proposed CA-AI framework achieves higher detection accuracy, lower latency, improved throughput, and stronger fault isolation than conventional centralized and thread-based IDS architectures. These findings indicate that concurrency management should be considered an integral architectural component, rather than merely an implementation detail, for scalable real-time SDN intrusion detection.

For latency, the CA-AI IDS achieved an average of 62 ms compared with 146 ms for the centralized IDS baseline, corresponding to an approximately 57.5% reduction. The proposed concurrency-oriented architecture has shown that per-flow isolation based on actors, load-sensitive workflow scheduling, and distributed asynchronous learning have a better performance than the traditional centralized and thread-based architectures[37]. The observed throughput improvements under increasing concurrency that have been observed are due to reduced shared-memory contention and asynchronous per-flow processing. This is possible through dynamic scheduling of the high-risk flows, which results in low latency, and through the use of fine-grained fault isolation and fast actor recovery, which leads to high reliability.

The proposed framework has not yet been evaluated in a production-scale SDN deployment. In particular, the authors did not evaluate experimental controller switch interactions, OpenFlow message exchanges, P4-programmable data planes, and SD-WAN infrastructures. Rather, the work is focused on validating the concurrency-aware architectural concepts of the framework through the use of benchmark intrusion detection datasets and a controlled workload. An operational SDN deployment evaluation of the framework is still important research direction. Therefore, the current results should be interpreted as architectural proof-of-concept evidence demonstrating the effectiveness of concurrency-aware design principles rather than full deployment readiness in operational SDN infrastructures.

Although the findings are also persuasive, it is worth mentioning that the research has certain study limitations: the experiments are carried out in a controlled SDN-based testbed using certain benchmark datasets (NSL-KDD, CIC-IDS2017, UNSW-NB15). Under different platforms of heterogeneous hardware, larger distributed controllers, or other traffic models, performance might be different. The framework should be tested in future research in order to confirm that it holds true in such far-fetched situations.

Another limitation of the current study is that a complete experimental ablation analysis was not conducted. Although the contribution of individual architectural components was qualitatively assessed through subsystem-specific performance metrics, the isolated quantitative impact of each component under controlled removal experiments was not evaluated. A comprehensive ablation study investigating the independent and combined effects of actor-based concurrency, workflow scheduling, asynchronous inference, and monitoring feedback mechanisms represents an important direction for future work.

In contrast to the existing integration-oriented AI frameworks, the CA-AI framework has a unique combination of concurrency-conscious design, workflow-based adaptive scheduling, and asynchronous learning, whereas the traditional frameworks usually use a layered pipeline, coarse-grained fault isolation, and synchronous learning execution. This co-designed integration delivers stable throughput growth under increasing concurrency, predictable latency behavior within the evaluated experimental environment, and resiliency to high-concurrency workloads, This supports the potential applicability of the framework to real-time, high-concurrency environments.

These findings indicate that in critical AI systems, concurrency, scheduling, and reliability software, which enable the system, ought to be designed alongside the learning model itself to guarantee production quality output. Overall, the results indicate that architectural awareness is just as significant as model accuracy in high-performance, real-time AI.

Conclusion

Concurrency-Aware Artificial Intelligence (CA-AI) framework is able to prove that co-designing deep learning models using a five-layered architectural model that unites Actor-Based Concurrency Management, Workflow-Oriented Scheduling, Agent-Based Parallel Feature Extraction, and Monitoring-Driven Feedback Loops is necessary in modern software systems. The framework achieved up to 2.1 × throughput improvement compared with the centralized baseline**,** while maintaining approximately 1.19 × throughput growth as concurrency increased from 10,000 to 100,000 flows. A comprehensive experimental ablation study to quantify the isolated and combined effects of individual architectural components remains an important direction for future research. This

concurrency-aware design, which is proven to achieve a high detection rate (up to 98.9% with multiple datasets) as well as give stable throughput growth under increasing concurrency (2.1 × throughput improvement) and a reduction in average latency from 146 ms for the centralized IDS baseline to 62 ms for CA-AI, corresponding to an approximately 57.5% reduction and reliability (99.6% system availability) under heavy concurrent workloads, is proven by experimental assessment, particularly as an architectural validation of SDN-oriented intrusion detection scenarios evaluated through benchmark datasets and controlled experimental workloads.

Future work

The discoveries of distributed trust and consistency in systems based on block chains in digital twins promote decentralized and fault-isolated architectural frameworks of real-time AI systems even more[38]. The future work aims at developing additional capabilities and strength of the CA-AI framework. The main improvements are to inject more modern temporal learning models, namely transformer-based streaming models, to obtain an analysis based on more complex temporal dependencies of network traffic. The team shall apply federated learning to various domains of Software-Defined Network (SDN) to increase the realm of deployment and privacy of data. Lastly, to have the greatest confidence in the integrity of the system, the framework will be used to formally verify the interactions of the actors in order to mathematically demonstrate the robustness and reliability of the concurrency control and fault isolation mechanisms. Future work will focus on conducting a comprehensive ablation study to quantify the contribution of each architectural component and better understand their combined impact on system performance.

Author contributions R.Kanimozhi: Contributed to conceptualization, problem formulation, system architecture design, methodology, investigation, algorithm development, implementation, coding, testing, validation, performance evaluation, result analysis, visualization, and preparation of the original draft, as well as critical review and editing of the manuscript.V.Padmavathi: Performance evaluation, result analysis as well as critical review and editing of the manuscript.M.Latha: Visualization, critical review and editing of the manuscript.

Data availability The datasets used and analysed during the current study available from the corresponding author on reasonable request.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Kanimozhi, R., Suhasini, A., Padmavathi, V., Gothainayagi, S. & Jayabharathi, B. Encrypting data before moving it to the cloud using Specter encryption. *Indian J. Sci. Technol.* **8**, 635 (2015).
2. Khattak, S. B. A. et al. Performance evaluation of an IEEE 802.15.4-based thread network for efficient Internet of Things communications in smart cities. *Appl. Sci.* **13**, 7745 (2023).

3. Sharafzadeh, E., Abdous, S. & Ghorbani, S. Understanding the impact of host networking elements on traffic bursts. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)* 237–254 (2023).
4. Sychowiec, J. & Zieliński, Z. A blockchain-based framework for secure data stream dissemination in federated IoT environments. *Electronics* **14**, 2067 (2025).
5. Capel, M. I. Algorithms and process synchronisation mechanisms based on shared memory. In *Concurrent and Real-Time Programming: Principles and Applications* 69–160 (Springer, 2025).
6. Karanasos, K. et al. Mercury: Hybrid centralized and distributed scheduling in large shared clusters. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)* 485–497 (2015).
7. Xie, X. Dynamic resource allocation of reinforcement learning based on neural networks in software defined networks. *Proc. Comput. Sci.* **243**, 834–841 (2024).
8. Shrivastwa, R. R. et al. An embedded AI-based smart intrusion detection system for edge-to-cloud systems. In *International Conference on Cryptography, Codes and Cyber Security* 20–39 (Springer, 2022).
9. Zhang, X. et al. Towards risk-aware artificial intelligence and machine learning systems: An overview. *Decis. Support Syst.* **159**, 113800 (2022).
10. Panchumarthy, R. & Benala, T. R. An overview of AI workload optimization techniques. In *Boosting Software Development Using Machine Learning* 269–299 (2025).
11. Choudhary, M. K. & Mishra, P. A. K. Optimization accuracy of intrusion detection system using LSTM deep learning technique. *Int. J. Res. Technol.* **13**, 272–279 (2025).
12. Hosawi, A. & Stone, R. Enhancing trust in human–AI collaboration: A conceptual review of operator–AI teamwork. *Int. J. Adv. Comput. Sci. Appl.* **16**, 356 (2025).
13. Al-Ibraheemi, F. A. et al. Intrusion detection in software-defined networks: Leveraging deep reinforcement learning with graph convolutional networks for resilient infrastructure. *Fusion Pract. Appl.* **8**, 78–87. https://doi.org/10.54216/FPA.150107 (2024).
14. Tang, T. A. et al. DeepIDS: Deep learning approach for intrusion detection in software-defined networking. *Electronics* **9**, 1533 (2020).
15. Choraś, M. & Pawlicki, M. Intrusion detection approach based on optimised artificial neural network. *Neurocomputing* **452**, 705–715 (2021).
16. Maddireddy, B. R. & Maddireddy, B. R. Adaptive cyber defense: Using machine learning to counter advanced persistent threats. *Int. J. Adv. Eng. Technol. Innov.* **1**, 305–324 (2023).
17. Anwar, R., Kwon, J.-W. & Kim, W.-T. A deep reinforcement learning-based concurrency control of federated digital twin for software-defined manufacturing systems. *Appl. Sci.* **15**, 8245 (2025).
18. Kanimozhi, R., Mary, M. A. S. & Johnson, S. Stabilizing distributed OLTP under high contention via measurement-driven adaptive concurrency control. *Sci. Rep.* https://doi.org/10.1038/s41598-026-60165-x (2026).
19. Datta, D. & Gordon, M. S. A massively parallel implementation of the CCSD(T) method using the resolution-of-the-identity approximation and a hybrid distributed/shared memory parallelization model. *J. Chem. Theory Comput.* **17**, 4799–4822 (2021).
20. Mansoor, A. et al. A deep learning-based approach for detecting DDoS attack on software-defined networking controller. *Systems* **11**, 296 (2023).
21. Kanimozhi, R. Adaptive and intelligent framework of data protection techniques for cloud storage. *Int. J. Cloud Comput.* **8**, 50 (2019).
22. Caldwell, S., Garnock-Jones, T. & Felleisen, M. Conversational concurrency with dataspaces and facets. *Proc. ACM Program Lang.* **10**, 2:1–2:37 (2025).
23. Karsai, G. et al. Distributed and managed: Research challenges and opportunities of the next generation cyber-physical systems. In *2014 IEEE 17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing* 1–8 (IEEE, 2014).
24. Michelon, G. K. et al. Analysis and propagation of feature revisions in preprocessor-based software product lines. In *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* 284–295 (IEEE, 2023). https://doi.org/10.1109/SANER56733.2023.00035
25. Fleuren, T., Götze, J. & Müller, P. Workflow skeletons: Increasing scalability of scientific workflows by combining orchestration and choreography. In *Proceedings of 9th IEEE European Conference in Web Services (ECOWS)* 106 (IEEE, 2011). https://doi.org/10.1109/ECOWS.2011.26
26. Guan, W. et al. An intelligent data delivery service for and beyond the ATLAS experiment. *EPJ Web Conf.* **251**, 02007 (2021).
27. Van den Heuvel, M. M. H. P., Bril, R. J., Schiemenz, S. & Hentschel, C. Dynamic resource allocation for real-time priority processing applications. *IEEE Trans. Consum. Electron.* **56**, 879–887 (2010).
28. Bansal, P. AI pattern recognition and its features. *Int. J. Eng. Adv. Technol.* **14**, 18–25 (2025).
29. Punniyamoorthy, V. et al. AI-driven cloud resource optimization for multi-cluster environments. Preprint at https://arxiv.org/abs/2512.24914 (2025).

30. Adam, M. S. et al. Neural-guided adaptive clustering for UAV-based user grouping in 5G/6G post-disaster networks. *Drones* **9**, 731 (2025).
31. Taheri, R. et al. Explainable AI for federated learning-based intrusion detection systems in connected vehicles. *Electronics* **14**, 4508 (2025).
32. Hu, X. et al. EIDS: A cloud intrusion detection system with high performance and maintainability. *ACM Trans. Comput. Syst.* **44**(14), 1–29. https://doi.org/10.1145/3787966 (2026).
33. Lim, D. et al. S-iNAS: A performance-centric SRN modeling and analysis of time- and event-based scaling strategies in Ceph-based industrial network-attached storage. *J. Netw. Syst. Manag.* **34**, 34 (2026).
34. Zhuang, Q. et al. GeoTP: Latency-aware geo-distributed transaction processing in database middlewares. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)* 433–445 (IEEE, 2025).
35. Li, H., Wang, S., Tan, G. & Duan, X. Cost-efficient and topology-aware scheduling algorithms in distributed stream computing systems. *Future Gener. Comput. Syst.* **170**, 108340 (2025).
36. Kanimozhi, R. & Padmavathi, V. A unified adaptive deterministic concurrency control framework for distributed systems. *Concurr. Comput. Pract. Exp.* **38**, e70638 (2026).
37. Haller, P. & Odersky, M. Scala actors: Unifying thread-based and event-based programming. *Theor. Comput. Sci.* **410**, 202–220 (2009).
38. McDonald, M. et al. Enhancing healthcare collaboration through blockchain-enabled digital twin integration. In *Human-Centric Integration of 6G-Enabled Technologies for Modern Society* 315–326 (Academic Press, 2025). https://doi.org/10.1016/B978-0-443-27434-3.00021-0
39. Wu, Z. et al. Energy-aware mapping and scheduling strategies for real-time workflows under reliability constraints. *J. Parallel Distrib. Comput.* **176**, 1–16 (2023).
40. Yang, W. & Liu, Z. Efficient vehicular edge computing: An asynchronous federated and deep reinforcement learning approach for content caching in VEC. *IEEE Access* **12**, 13196–13212 (2024).
41. Tyurin, A., Spiridonov, A. & Rudenko, V. Asynchronous policy gradient aggregation for efficient distributed reinforcement learning. Preprint at https://doi.org/10.48550/arXiv.2509.24305 (2025).
42. Stylianopoulos, C. et al. Delegation sketch: a parallel design with support for fast and accurate concurrent operations. In *Proceedings of the Fifteenth European Conference on Computer Systems* 1–16 (2020).
43. Kanimozhi, R. & Ramesh, P. S. Deep reinforcement learning-based intrusion detection scheme for software-defined networking. *Sci. Rep.* **15**, 38827. https://doi.org/10.1038/s41598-025-24869-w (2025).

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

R. Kanimozhi[1] · V. Padmavathi[2] · M. Latha[3]

✉ R. Kanimozhi
kanivedhabackup@gmail.com

[1] Department of Artificial Intelligence and Data Science, A.V.C. College of Engineering, Mayiladuthurai, TamilNadu, India

[2] Department of Information Technology, A.V.C. College of Engineering, Mayiladuthurai, TamilNadu, India

[3] Department of Electrical and Electronics Engineering, A.V.C. College of Engineering, Mayiladuthurai, TamilNadu, India